# Manual

## Development Suite MES Cockpit Services

## MC-DSCS 3.2

Version 1.1.23049

Last changed on: 01.09.2020

## Copyright

# Contents

# 1 Development Suite MES-Cockpit-Services

MES Development Suite offers functions for customizing the MES-Cockpit 3.1 according to the customer's requirements. This document describes the functions provided by the customizing tool MES-Cockpit (MC-DSCS).

Default *.qvw files and scripts must not be changed. Unless otherwise specified, changes must always be made in customer-specific files.

All modifications described here can only be made if the user has purchased the customizing tool (MC-DSCS). It is recommended to complete the relevant customizing training course (CUT-MSC).

## 1.1 Advanced Analysis Options

**Overview of benefits**

By default, there are already pre-defined evaluations for the Performance Analysis and Production Monitoring that can be applied by users. These provided options may be customized and modified.

MES-Cockpit uses QlikView to visualize data in the Performance Analysis and Performance Monitoring.

### Requirements

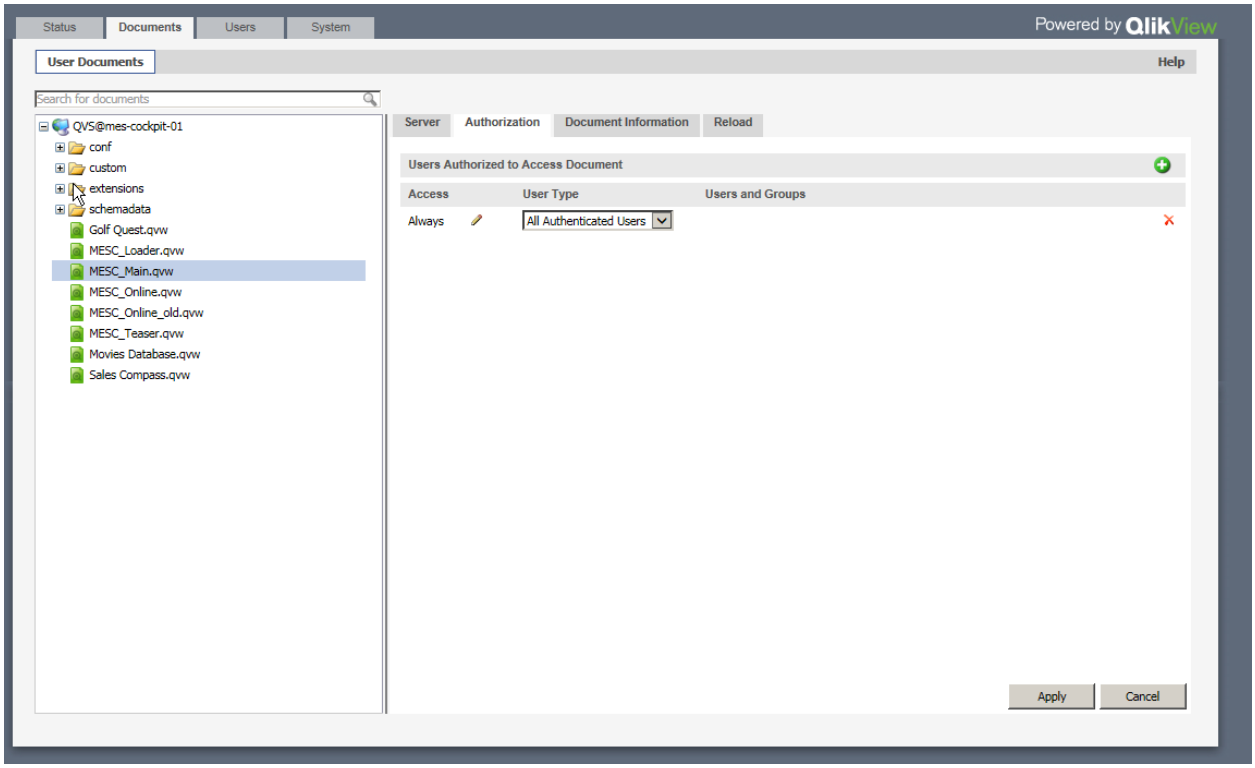
The original QlikView document can be found in the following path on the respective MES-Cockpit server:

Production Monitoring: `C:\ProgramData\QlikTech\Documents\MESC_Online.qvw`
Performance Analysis: `C:\ProgramData\QlikTech\Documents\MESC_Main.qvw`

The original document must not be changed. Provided that modifications are required, the file has to be copied and renamed. Example: `MESC_Online_<Customer's ID>.qvw`

Only then changes to this customer-specific file can be made. The options provided in QlikView are described in the document: MESC_DevelopmentSuiteQV.pdf

If a new qvw file is copied or created, it must be accessible for authenticated users. To do so, the created file has to be assigned to the user type "All Authenticated Users" in Document --> Authorization in the management console of QlikView on the MES-Cockpit server.




## 1.2 Modifying Basic KPIs

**Overview of benefits**

Basic KPIs used for calculations and display in MES-Cockpit can be customized, i.e. further information may be added and/or they may also be restricted according to the customer's requirements.

Basic KPIs encompass all data exported from connected HYDRA systems and saved as *.xml file on the MES-Cockpit server for the calculation of KPIs. Customizing provides the following options:

- Additional data of already integrated Web Services for existing objects
- Integration of customer-specific basic KPIs for existing data types

By default, the following fields are exported:

**Object "workplace":**

*MasterConfiguration:*

| Object | Data type | Web service | Fields |
|---|---|---|---|
| **Workplace** | Master data | BOResourceList | resource.id<br>resource.designation<br>resource.short_name<br>resource.company |

| | | | |
|---|---|---|---|
| | | | resource.group<br>resource.costcenter<br>resource.blocked |
| | Online data | BOResourceList | "resource.id",<br>"resource.short_name",<br>"resource.group",<br>"resource.costcenter",<br>"resource.act.status.text",<br>"resource.act.status",<br>"resource.act.status.color",<br>"resource.act.start_of_status",<br>"resource.act.shift.total_time",<br>"resource.cycle.target",<br>"resource.cycle.actual",<br>"resource.act.shift.yield.primary",<br>"resource.act.shift.scrap.primary",<br>"resource.act.shift.rpa1",<br>"resource.act.shift.rpa2",<br>"resource.act.shift.rpa3",<br>"resource.act.shift.rpa4",<br>"resource.act.shift.rpa5",<br>"resource.act.shift.rpa6",<br>"resource.act.shift.rpa7",<br>"resource.act.shift.rpa8",<br>"resource.act.shift.rpa9",<br>"resource.act.shift.rpa10",<br>"resource.act.shift.rpa11",<br>"resource.act.shift.rpa12",<br>"resource.act.strokes.total",<br>"resource.act.strokes.yield" |
| | Key figure | Workplacebooking.list | "resource.id",<br>"workplacebooking.shift.date",<br>"workplacebooking.shift.number",<br>"workplacebooking.rpa.number",<br>"workplacebooking.yield.base",<br>"workplacebooking.yield.primary",<br>"workplacebooking.yield.secondary",<br>"workplacebooking.yield.tertiary",<br>"workplacebooking.scrap.base",<br>"workplacebooking.scrap.primary",<br>"workplacebooking.scrap.secondary",<br>"workplacebooking.scrap.tertiary",<br>"workplacebooking.rework.base",<br>"workplacebooking.rework.primary",<br>"workplacebooking.rework.secondary",<br>"workplacebooking.rework.tertiary",<br>"workplacebooking.problem.base",<br>"workplacebooking.problem.primary",<br>"workplacebooking.problem.secondary",<br>"workplacebooking.problem.tertiary",<br>"workplacebooking.strokes.total",<br>"workplacebooking.status",<br>"workplacebooking.status_duration",<br>"workplacebooking.cycle.target",<br>"workplacebooking.partitioning",<br>"resource.pulse_factor.target"<br><br>**Calculated fields from MDC:**<br>w.status.class |

| | | | |
|---|---|---|---|
| | | | *Total duration accrued for a status of the relevant status class*<br>w.oeee_arith<br>*Performance is calculated on the MDE log record level. In this regard, all data records collected during the "production" status (RPA11) are used. Performance is calculated for each log record. For a compressed presentation in evaluations, these individual performance values are weighted. Weighting is based on production time (RPA11). The value exported for each workplace and shift represents the sum of weighted performance values.* |
| **Operation** | Master data | BOOperation.list | "order.id",<br>"operation.id",<br>"operation.costcenter",<br>"operation.article",<br>"operation.customerdesignation",<br>"operation.articledesignation",<br>"operation.ordertype",<br>"operation.plan.yield.base",<br>"operation.plan.yield.primary",<br>"operation.plan.yield.secondary",<br>"operation.plan.yield.tertiary",<br>"operation.act.yield.base",<br>"operation.act.yield.primary",<br>"operation.act.yield.secondary",<br>"operation.act.yield.tertiary",<br>"operation.act.scrap.base",<br>"operation.act.scrap.primary",<br>"operation.act.scrap.secondary",<br>"operation.act.scrap.tertiary",<br>"operation.act.rework.base",<br>"operation.act.rework.primary",<br>"operation.act.rework.secondary",<br>"operation.act.rework.tertiary",<br>"operation.act.problem.base",<br>"operation.act.problem.primary",<br>"operation.act.problem.secondary",<br>"operation.act.problem.tertiary",<br>"operation.processing_time" |
| | Online data | BOOperationListCurrentlyLoggedOn | "operation.id",<br>"operation.article",<br>"operation.articledesignation",<br>"order.id",<br>"bookingrelation.logon_ts",<br>"operation.designation",<br>"operation.plan.yield.primary",<br>"operation.act.yield.primary",<br>"operation.act.scrap.primary",<br>"bookingrelation.workplace",<br>"bookingrelation.rpa1",<br>"bookingrelation.rpa2",<br>"bookingrelation.rpa3",<br>"bookingrelation.rpa4",<br>"bookingrelation.rpa5",<br>"bookingrelation.rpa6", |

| | | | |
|---|---|---|---|
| | | | "bookingrelation.rpa7",<br>"bookingrelation.rpa8",<br>"bookingrelation.rpa9",<br>"bookingrelation.rpa10",<br>"bookingrelation.rpa11",<br>"bookingrelation.rpa12",<br>"operation.act.first_logon_ts",<br>"operation.act.last_logoff_ts",<br>"operation.act.last_interruption_ts",<br>"operation.act.status",<br>"operation.act.status.textnumber",<br>"operation.plan.start_ts",<br>"operation.plan.end_ts",<br>"operation.earliest_start_ts",<br>"operation.latest_end_ts",<br>"operation.scheduled_start_ts",<br>"operation.scheduled_end_ts",<br>"operation.setup_time",<br>"operation.processing_time" |
| | Key figure | DCAdeLogRecord.list | "operation.id",<br>"orderbooking.shift.start_ts",<br>"orderbooking.shift.number",<br>"orderbooking.rpa1",<br>"orderbooking.rpa2",<br>"orderbooking.rpa3",<br>"orderbooking.rpa4",<br>"orderbooking.rpa5",<br>"orderbooking.rpa6",<br>"orderbooking.rpa7",<br>"orderbooking.rpa8",<br>"orderbooking.rpa9",<br>"orderbooking.rpa10",<br>"orderbooking.rpa11",<br>"orderbooking.rpa12",<br>"orderbooking.yield.base",<br>"orderbooking.yield.primary",<br>"orderbooking.yield.secondary",<br>"orderbooking.yield.tertiary",<br>"orderbooking.scrap.base",<br>"orderbooking.scrap.primary",<br>"orderbooking.scrap.secondary",<br>"orderbooking.scrap.tertiary",<br>"orderbooking.rework.base",<br>"orderbooking.rework.primary",<br>"orderbooking.rework.secondary",<br>"orderbooking.rework.tertiary",<br>"orderbooking.problem.base",<br>"orderbooking.problem.primary",<br>"orderbooking.problem.secondary",<br>"orderbooking.problem.tertiary",<br>"orderbooking.labor_utilization" |
| **Order** | Master data | BOOrderOverview | "order.id",<br>"order.costobject",<br>"order.article",<br>"order.projectnumber",<br>"order.customerdesignation",<br>"order.salesorder",<br>"order.articledesignation",<br>"order.type", |

| | | | |
|---|---|---|---|
| | | | "order.act.status.text",<br>"order.act.status.color",<br>"order.last_op_logoff_ts_calc",<br>"order.no_recordable_op",<br>"order.no_finished_op",<br>"order.act.rpa1",<br>"order.act.rpa2",<br>"order.act.rpa3",<br>"order.act.rpa4",<br>"order.act.rpa5",<br>"order.act.rpa6",<br>"order.act.rpa7",<br>"order.act.rpa8",<br>"order.act.rpa9",<br>"order.act.rpa10",<br>"order.act.rpa11",<br>"order.act.rpa12",<br>"order.act.yield.base",<br>"order.act.yield.primary",<br>"order.act.yield.secondary",<br>"order.act.yield.tertiary",<br>"order.act.scrap.base",<br>"order.act.scrap.primary",<br>"order.act.scrap.secondary",<br>"order.act.scrap.tertiary",<br>"order.act.rework.base",<br>"order.act.rework.primary",<br>"order.act.rework.secondary",<br>"order.act.rework.tertiary",<br>"order.act.problem.base",<br>"order.act.problem.primary",<br>"order.act.problem.secondary",<br>"order.act.problem.tertiary",<br>"order.plan.yield.base",<br>"order.plan.yield.primary",<br>order.plan.yield.secondary",<br>"order.plan.yield.tertiary",<br>"order.plan.lead_time",<br>"order.plan.total_setup_time",<br>"order.plan.processing_time",<br>"order.plan.execution_time",<br>"order.act.retention_period",<br>"order.act.lead_time",<br>"order.act.labor_utilization",<br>"order.act.processing_time",<br>"order.act.occupancy_time",<br>"order.act.standstill_period",<br>"order.act.setup_time",<br>"order.act.wait_time",<br>"order.first_op_logon_ts",<br>"order.last_op_logoff_ts",<br>"order.scheduled_start_ts",<br>"order.scheduled_end_ts",<br>"order.plan.labor_utilization",<br>"order.ordertype" |
| | Online data | BOOrderOverview | "order.id",<br>"order.article",<br>"order.articledesignation",<br>"order.salesorder", |

| | | | |
|---|---|---|---|
| | | | "order.act.status.led",<br>"order.act.status.text",<br>"order.act.status.color",<br>"order.act.last_posting_ts",<br>"order.earliest_start_ts",<br>"order.scheduled_end_ts",<br>"order.latest_end_ts",<br>"order.scheduled_start_ts",<br>"order.plan.yield.base",<br>"order.act.yield.base",<br>“order.act.scrap.base",<br>"order.plan.lead_time",<br>"order.plan.total_setup_time",<br>"order.plan.processing_time",<br>"order.plan.labor_utilization",<br>"order.plan.execution_time",<br>"order.act.retention_period",<br>"order.act.lead_time",<br>"order.act.setup_time",<br>"order.act.processing_time",<br>"order.act.standstill_period",<br>"order.act.occupancy_time",<br>"order.act.labor_utilization" |
| **Operation_scrap** | Key figure | DCAdeLogRecord.list | Scrap per scrap reason (orderbooking.scrap.reason) |

### 1.2.1 Additional data of already integrated Web Services for existing objects

The following definition can be used to export further data for existing objects, such as for the "workplace" object:

By default, the single DataGetter functions define which data is exported via WebServices. This definition may be overwritten by customer-specific definitions in the config.xml file of MDC.

A template that may be changed is included in the MDC templates. MDC templates can be found here: `c:\ProgramData\mpdv\mdc\templates\`

The MDC templates include DataGetter functions for individual objects. Subject to the area from which data is to be taken, definitions have to be made in the xml file which also results in the data to be filed accordingly.

The following definitions can be made for existing objects:

| Object | DataGetter | Data type | Web service | Config.xml |
|---|---|---|---|---|
| **Workplace** | WorkplaceDataGe | Master data | BOResourceList | MasterResultParameter |

| | | | | |
|---|---|---|---|---|
| | tter | Online data | BOResourceList | OnlineDataResultParameter |
| | | Key figure | Workplacebooking.list | KeyFigureResultParameter |
| **Operation** | OperationDataGetter | Master data | BOOperation.list | MasterResultParameter |
| | | Online data | BOOperationListCurrentlyLoggedOn | OnlineDataResultParameter |
| | | Key figure | DCAdeLogRecord.list | KeyFigureResultParameter |
| **Operation_scrap** | OperationDataGetter | Key figure | mescarchiveloadoperationscrap | KeyFigureResultParameter |
| **Order** | OrderDataGetter | Master data | BOOrderOverview | MasterResultParameter |

All data from the "Keyfigure" area are exported as a total for each shift.

Customizations have to be inserted in the following MDC configuration file: `C:\ProgramData\mpdv\mdc\config.xml`

**Example:**

```
<ComponentConfiguration Name="WorkplaceDataGetter">

      <Parameter Name="MasterResultParameter"
      Value="resource.userfield02,resource.id,resource.designation,resource.short_name,
      resource.company,resource.group,resource.costcenter,resource.blocked"/>

      <Parameter Name="KeyFigureResultParameter" Value="<New field>,<List of key figures>"/>

      <Parameter Name="OnlineDataResultParameter" Value="<New field>,<List of online data>"/>

</ComponentConfiguration>
```

Please note: If a DataGetter function is integrated, only the fields listed in "Value" are requested by the WebService. This means: if a field is to be added all fields that should be exported for this object should be listed. In addition, every time data has been changed, the MDC service has to be restarted in MES-Cockpit server services.

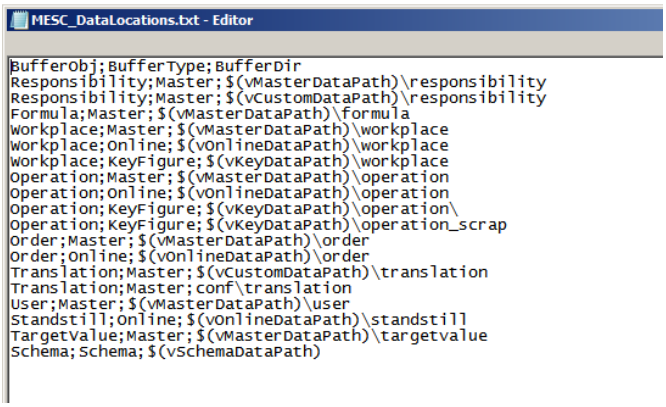
### 1.2.3 Integration of customer-specific basic KPIs for existing data types

The defined XML file interface described in the customizing document entitled MESC_DevelopmentSuiteInterface.pdf can be used to integrate customer-specific basic KPIs.[1]

The requirements for using the XML file interface are:

- The structure described in the relevant document has been adhered to
- Data is transferred for existing data types
- Meta data/master data of objects are included
- Data to be transferred has to be filed in a new directory on the MES-Cockpit server. The directory name starts with the object name and an underscore character "_", e.g. Workplace_

The defined storage locations can be found in the configuration file MESC_DataLocations.txt (c:\ProgramData\QlikTech\Documents\conf\)




## 1.3 Modifications to the Home Screen

**Overview of benefits**

The buttons displayed in the home screen of MES-Cockpit can be customized and/or additional buttons/icons may be added.

### 1.3.1 Customizing the first page of MES-Cockpit

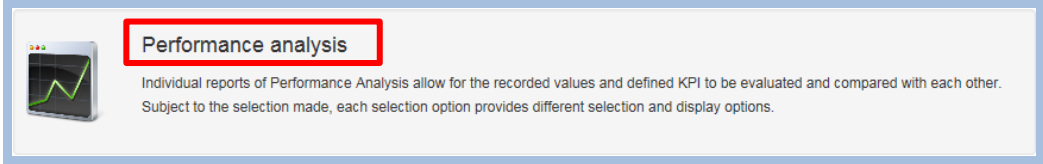
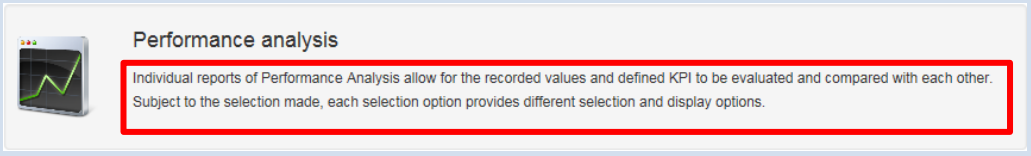
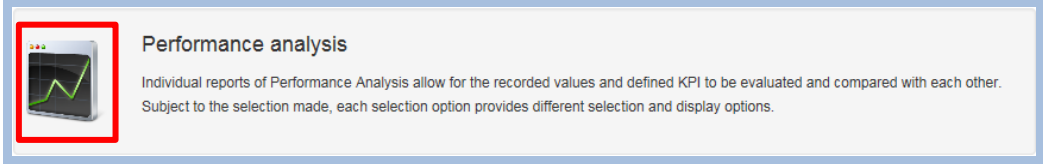
The file C:\inetpub\wwwroot\SMAMESC\Areas\Mesc\menu.xml includes the buttons/icons displayed in the home screen.

By default, the following buttons are shown in MES-Cockpit:

[1] Only customer-specific basic KPIs relating to HYDRA MES data may be integrated.

- Performance Analysis
- Production Monitoring
- Workplaces/ machines
- KPI monitor
- Contacts
- Messages listing

The following TAGs have to be defined in the menu.xml for each button:

| TAG | Description |
|---|---|
| **Label** | LanguageKey displayed as caption in the button<br>  |
| **Description** | LanguageKey of the brief description that is to be displayed in the button<br>  |
| **Icon** | Path leading to the displayed icon of the button. The file has to be stored here: C:\inetpub\wwwroot\SMAMESC\Content\img<br>  |
| **Action** | Function call to be started via the button. |

Default sample configuration:

```
<?xml version="1.0"?>
<ArrayOfButtonView xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
```

```
<ButtonView>
<Label>lkMescPerformanceAnalysis</Label>
<Description>lkMescPerformanceAnalysisDescription</Description>
<Icon>/Content/img/Stats.png</Icon>
<Action> <Target>location='../mesc/qvcontainer?qvItem=MESC_Main'</Target>
</Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>

<ButtonView>
<Label>lkMescOnlineMonitor</Label>
<Description>lkMescOnlineMonitorDescription</Description>
<Icon>/Content/img/WebSystem.png</Icon>
<Action> <Target>location='../mesc/qvcontainer?qvItem=MESC_Online'</Target>
</Action> <DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>

<ButtonView>
<Label>lkWorkplaceOverview</Label>
<Description>lkWorkplaceOverviewSmaDescription</Description>
<Icon>/Content/img/Generators.png</Icon>
<Action> <Target>location='../resource/resource'</Target> </Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>

<ButtonView>
<Label>lkKeyMonitor</Label>
<Description>lkKeyMonitorSmaDescription</Description>
<Icon>/Content/img/ReportsPieChart.png</Icon>
<Action> <Target>location='../oee/kzm'</Target> </Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>

<ButtonView> <Label>lkContactPersons</Label>
<Description>lkContactPersonsDescription</Description>
<Icon>/Content/img/BusinessPartnersBlueMaleRedFemale.png</Icon> -<Action>
<Target>location='../ContactPersons/ContactPersons'</Target> </Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>
```

```
<ButtonView> <Label>lkMessageList</Label>
<Description>lkMessageListSmaDescription</Description>
<Icon>/Content/img/Generators.png</Icon> -<Action>
<Target>location='../MaintenanceManagement/MessageList'</Target> </Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>

</ArrayOfButtonView>
```

In addition to the configuration in the menu.xml file, the sub-folder "profiles" (C:\inetpub\wwwroot\SMAMESC\Areas\Mesc\Profiles) includes a configuration file for each integrated qvw file. This file includes the qvw file to be called up for the entry in the home screen according to the following pattern:

Example for MESC_Main.qvw:

```
<QvAppConfig xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

<Server>http://swe-cbu-01</Server>
<Document>MESC_Main.qvw</Document>
<ApplicationName>lkMescPerformanceAnalysis</ApplicationName>
<ApplicationId>PerformanceAnalysis</ApplicationId>

</QvAppConfig>
```

## 1.3.2 Changes to existing buttons

If changes are required, they have to be made in a customer-specific menu_LOCAL.xml file. The entries from the original menu.xml file can be used as template. The file can be found here:

```
C:\inetpub\wwwroot\SMAMESC\Areas\Mesc\menu.xml
```

Available parameters and modification options are described in section 1.3.1.

## 1.3.3 Calling up a new evaluation/report with an existing button

If one of the existing QlikView files is to be changed, it has to be copied and renamed at first (e.g. MESC_Main_xxx.qvw). Then all changes/modifications can be carried out within the range of options provided by QlikView. The following measures have to be taken making sure that the new (changed) file is started and updated cyclically via the home screen instead of the previous QlikView file:

- Adjusting the relevant profiles file (C:\inetpub\wwwroot\SMAMESC\Areas\Mesc\Profiles) by defining the new document name, e.g. MESC_Main_xxx.qvw
  Example:

```
<QvAppConfig xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<Server>http://swe-cbu-01</Server>
<Document> MESC_Main_xxx.qvw </Document>
<ApplicationName>lkMescPerformanceAnalysis</ApplicationName>
<ApplicationId>PerformanceAnalysis</ApplicationId>
</QvAppConfig>
```

- Changing the MDC and transferring the reload section to the config.xml file of MDC:

```
- <ComponentConfiguration Class="mpdv.MachineDataCollector.ConnectorPlugin.Qlik.QmsCallerComponent" Name="ReloadOnline">
    <Parameter Name="QmsServiceUri" Value="http://[SERVER]:4799/QMS/Service"/>
    <Parameter Name="TaskName" Value="Reload of MESC_Online.qvw"/>
    <Parameter Name="TaskPassword" Value="mpdv"/>
    <Parameter Name="EventFilter" Value="onlinedata"/>
</ComponentConfiguration>
```

### 1.3.4 Integration and starting of a new, additional evaluation/report

When integrating a new qvw file, it can be created based on the existing application. Please proceed as follows:

By copying an existing QlikView file, a new one will be created, e.g. MESC_Main.qvw. Proceed as follows making sure this one will be displayed, called up and updated as an independent button/icon in the home screen in addition to the existing ones:

1. The new entry has to be added to the file menu_LOCAL.xml (C:\inetpub\wwwroot\SMAMESC\Areas\Mesc) displaying the buttons/icons. To do so, an already existing configuration for e.g. MESC_Main can be copied and adjusted accordingly.
   - 
```
<ButtonView>
<Label>lkxxx</Label>
<Description>lkxxxDescription</Description>
<Icon>/Content/img/xx.png</Icon>
<Action> <Target>location='../mesc/qvcontainer?qvItem=<Name of
the qvw-Datei withour Extension>'</Target> </Action>
<DoNotDissMissModal>false</DoNotDissMissModal>
</ButtonView>
```

**Please note:** The entered language key must be maintained in the relevant translation file and the defined icon must be created in the relevant path.
The name of the icon has to start with u_.

2. A new profile configuration file (C:\inetpub\wwwroot\SMAMESC\Areas\Mesc\Profiles) must also be created for the new qvw file. To do so, an existing configuration file can be copied, renamed and modified accordingly. Example:

```
<?xml version="1.0"?>
<QvAppConfig xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<Server>http://swe-cbu-01</Server>
<Document>MESC_Main_xxx.qvw</Document>
<ApplicationName>lkxxx</ApplicationName>
<ApplicationId>xxx</ApplicationId>
</QvAppConfig>
```

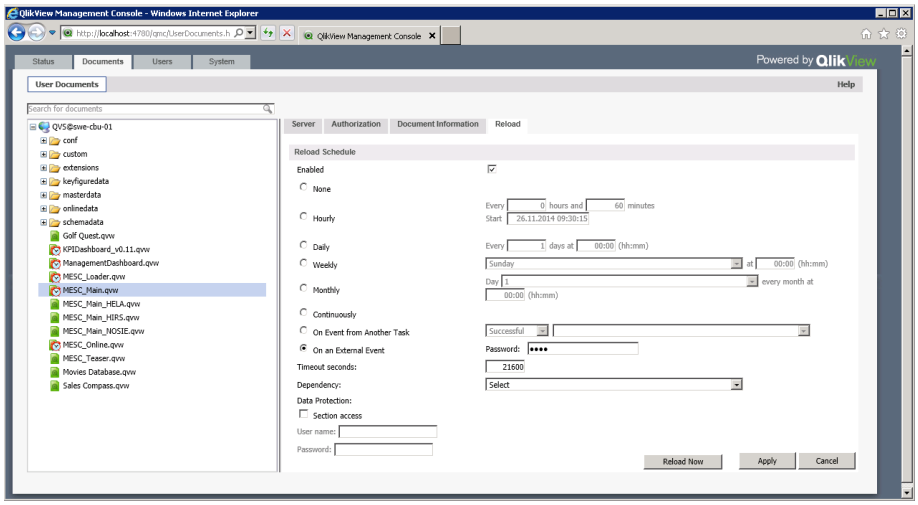
   A new "ApplicationID" must be assigned.

3. Organize reload

   The following alternatives can be configured in order to reload data and/or the qvw file including the data in cyclic intervals:

   - Definitions via the QlikView Management console

     A reload may be defined for a stored document.




     Either for a cyclic event, e.g. once a day at 7.00 a.m. or it is also possible to attach the reload to an "external event". In this case the user's password must be entered and reloading is triggered by the MDC. In this case, the following step must also be carried out:

   - Integrating the reload into the MDC

     Proceed as follows to trigger the reload process by MDC:

     - The following paragraph has to be entered in the config.xml file of MDC:

```
- <ComponentConfiguration Class="mpdv.MachineDataCollector.ConnectorPlugin.Qlik.QmsCallerComponent" Name="ReloadOnline">
    <Parameter Name="QmsServiceUri" Value="http://[SERVER]:4799/QMS/Service"/>
    <Parameter Name="TaskName" Value="Reload of MESC_Online.qvw"/>
    <Parameter Name="TaskPassword" Value="mpdv"/>
    <Parameter Name="EventFilter" Value="onlinedata"/>
</ComponentConfiguration>
```

     - Please note:
       - A new name must be entered

- The name of the new qvw file has to be entered in "Value" of the parameter "task name"
- The event filter has to be set accordingly: "online" for online data or "masterdata,keyfiguredata" for master data and key figure data

**Error! Reference source not found.**

## 1.4 Further customizing options

### 1.4.1 Customer-specific translations

**Overview of benefits**

The displayed texts can be customized for the single areas.

#### 1.4.1.1 Translations in Performance Analysis and Production Monitoring

All texts displayed in the applications can be renamed according to the customer's requirements by using a customized resource file.

To do so, the language keys (lk...) and relevant translations have to be entered in the Excel file MescDictionary.xlsm (.\conf\translation\MescDictionary.xlsm) and then the resource file has to be generated by clicking the "Export" button and filed here:

Default path: `C:\ProgramData\QlikTech\Documents\conf\translation`

Customized path: `C:\ProgramData\QlikTech\Documents\custom\translation`

Once the customer-specific resource file has been exported, the file .\custom\translation\translation.qvd has to be deleted.

The variable LOCALIZE can be used to access translations included in QV applications (example: $(LOCALIZE('lkYield'))). It replaces a LanguageKey by the translation defined in the user's language.

#### 1.4.1.2 Translations in the First Page of MES-Cockpit

All texts of the start page have to be maintained in the "SMA" area and/or they may be renamed according to the customer's requirements in a customer-specific resource file.

To do so, the language keys (lk...) and relevant translations have to be entered in the Excel file mpdvDictionaryCustomer.xltm and then the resource file has to be generated by clicking the "Export" button and filed here:

C:\inetpub\wwwroot\SMAMESC\Runtime\resources\standard\languages

## 1.4.2 Definition of refresh rates

Data exports from connected HYDRA systems are triggered in cyclic intervals. Different triggers are used depending on whether online data or basic KPIs are exported. The following cycles are defined by default:

- Exporting status information: every 1.5 minutes (90 seconds)
- Exporting basic KPIs: every 30 minutes (1800 seconds)

The MDC attempts to trigger the export based on the above-mentioned cycles. Reasons why triggering fails and no data is exported:

- If data is currently being exported (long export times), the new export process will not be started
- A new export process can only be initiated, once reloading of data to the qvw file has been finished
- Reloading of all qvw files must be completed before starting a new reload process, i.e. by default the files MESC_Main and MESC_Online - there are dependencies between both qvw files.

If the cycle is to be changed, the relevant part from the template file (C:\ProgramData\mpdv\mdc\templates\config_Mesc.xml) including the modified value must be copied into the config.xml file of MDC. Example:

| | |
|---|---|
| **Basic KPIs "order"** | <Parameter Name="[GetMasterDataOrder]" Value="1800" /> |
| **Target values** | <Parameter Name="[GetMasterDataTargetValue]" Value="1800" /> |
| **Responsibility areas** | <Parameter Name="[GetMasterDataResponsibility]" Value="1800" /> |
| **Formulas** | <Parameter Name="[GetMasterDataFormula]" Value="1800" /> |
| **Master data "operation"** | <Parameter Name="[GetMasterDataOperation]" Value="1800" /> |
| **Master data "workplace"** | <Parameter Name="[GetMasterDataWorkplace]" Value="1800" /> |
| **Basic KPIs "operation"** | <Parameter Name="[GetKeyFigureDataOperation]" Value="1800" /> |
| **Basic KPIs "OP scrap evaluation"** | <Parameter Name="[GetKeyFigureDataOperation_Scrap]" Value="1800" /> |

| | |
|---|---|
| **Basic KPIs "workplace"** | <Parameter Name="[GetKeyFigureDataWorkplace]" Value="1800" /> |
| **Online data "workplace"** | <Parameter Name="[GetOnlineDataWorkplace]" Value="90" /> |
| **Online data "operation"** | <Parameter Name="[GetOnlineDataOperation]" Value="90" /> |
| **Online data "order"** | <Parameter Name="[GetOnlineDataOrder]" Value="90" /> |
| **Online data "downtime hit list"** | <Parameter Name="[GetOnlineDataStandstill]" Value="90" /> |

If the section "TimedTrigger" is not included in the config.xml file, it has to be added. Please note in this context that the parameter "class" has to be removed.

## 1.4.3 Integration of new HYDRA systems

### 1.4.3.1 Integration of New HYDRA Systems into Performance Analysis and Production Monitoring

Connected systems from which data is exported can be found here:

```
c:\ProgrammData\mpdv\mdc\config.xml
```

For each connected HYDRA system there is an entry that is distinctly specified by the attribute "NumberOfLines" .

Each HYDRA system to be connected to MES-Cockpit must be entered in the configuration of MDC. The attribute "NumberOfLines" is to be set to the number of configured systems.

```
<ComponentConfiguration Name="WebServiceConnectorX">
    <Parameter Name="SystemName" Value="<Name>" />
    <Parameter Name="Host" Value="<Host>" />
    <Parameter Name="Port" Value="<Port>" />
    <Parameter Name="User" Value="<User>" />
    <Parameter Name="Pwd" Value="<Pwd>" />
</ComponentConfiguration>
```

A customer-specific identifier should be entered as the SystemName (maximum length 15 characters; [0-9a-zA-Z]). As an alternative, the host name can be entered.

**Please note:** If the configuration is changed and to enable these changes the MPDV MachineDataCollector service has to be restarted. Please also take into account that already exported data have actually been exported for the previously defined system. (Exported data include the name of the system they refer to. If this name is changed, please note that "old" data still include the "previous" name).

### 1.4.3.2 Integration of New HYDRA Systems in Production Information

All HYDRA systems to be evaluated in the Production Information area have to be entered in the following file:

```
…\SystemX\HyInstMgrDir\Instancerepo.properties
```

Example:
Configuration for the Hydra8 administration system "mesc-server:8080“ and the systems "hydra-server1:8080“, "hydra-server2:8082“:

```
=====================================
instance.name.1=MESC-ADMIN-SERVER

instance.name.2=HYDRA-SERVER
instance.name.3=HYDRA-SERVER2

MESC-ADMIN-SERVER.host.1=mesc-server
MESC-ADMIN-SERVER.port.1=8080
MESC-ADMIN-SERVER.id.1=1
MESC-ADMIN-SERVER.regserver.host.1=mesc-server
MESC-ADMIN-SERVER.regserver.port.1=6000
MESC-ADMIN-SERVER.description.1=MESC Admin

HYDRA-SERVER.port.1=8080
HYDRA-SERVER.host.1=hydra-server1
HYDRA-SERVER.id.1=1
HYDRA-SERVER.regserver.host.1=hydra-server1
HYDRA-SERVER.regserver.port.1=6000
HYDRA-SERVER.description.1=Hydra Germany

HYDRA-SERVER2.port.1=8081
HYDRA-SERVER2.host.1=hydra-server2
HYDRA-SERVER2.id.1=1
HYDRA-SERVER2.regserver.host.1=hydra-server2
HYDRA-SERVER2.regserver.port.1=6000
HYDRA-SERVER2.description.1=Hydra France
=====================================
```

## 1.5 Tips and tricks

### 1.5.1 Renaming of exported sites

When implementing and/or connecting HYDRA systems, a name is defined that will be used as site ID for exported data as of the first export. If another name should be displayed in MES-Cockpit this can be changed via the configuration file MESC_PlantMapping.txt.

This file can be found here:

`C:\ProgramData\QlikTech\Documents\conf\MESC_PlantMapping.txt`

This file maps the plant/site ID, i.e. the system name that is also exported and the newly defined name.

Example:

```
PlantId;PlantName
Plant1;Germany
Plant2;France
Plant3;China
```

### 1.5.2 Storage locations of values

Storage locations are configured in the following file in order for QlikView to "find" the exported data:

`C:\ProgramData\QlikTech\Documents\conf\MESC_DataLocations.txt`

Examples from the default file:

```
Operation;Master;$(vMasterDataPath)\operation

Operation;Online;$(vOnlineDataPath)\operation

Operation;KeyFigure;$(vKeyDataPath)\operation\
```

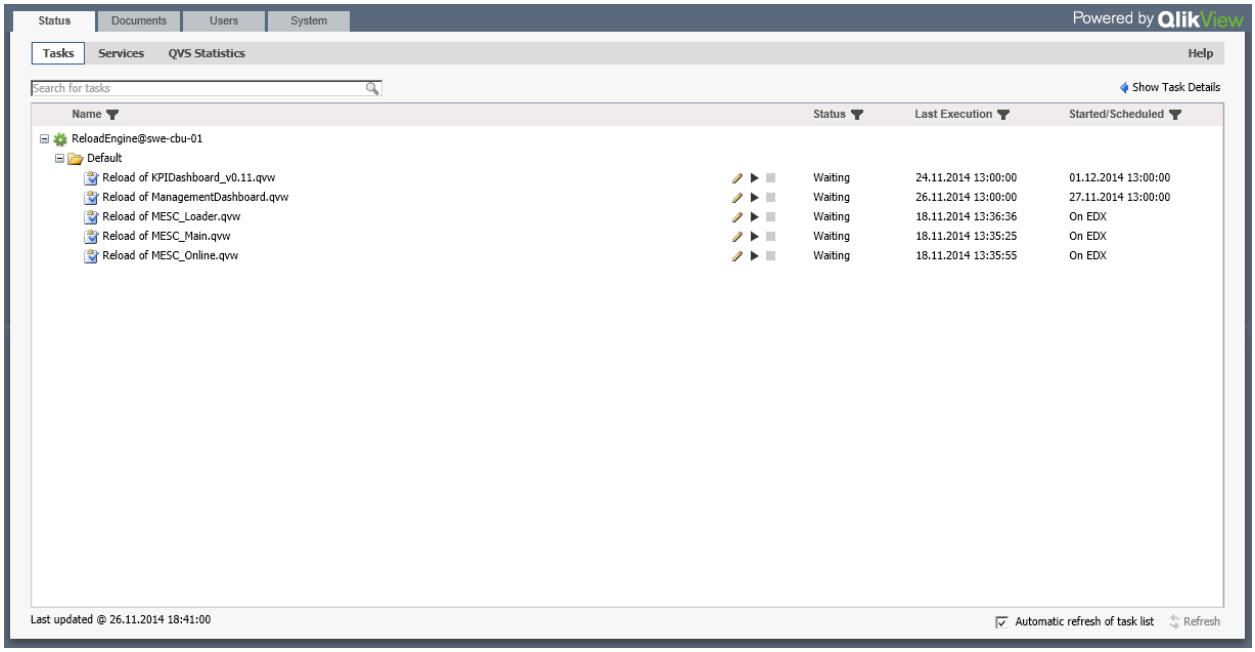
### 1.5.3 Defined variables used in QlikView

The following file includes the variables that can be used in and by QlikView:

`C:\ProgramData\QlikTech\Documents\conf\MESC_Variables.txt`

### 1.5.4 Last reload

The time stamp of the relevant QlikView file in the management console of QlikView shows the date and time the QlikView file was reloaded at last.


Status
Documents
Users
System
Powered by QlikView
Tasks
Services
QVS Statistics
Help
Search for tasks
Show Task Details
Name
Status
Last Execution
Started/Scheduled
ReloadEngine@swe-cbu-01
Default
Reload of KPIDashboard_v0.11.qvw
Waiting
24.11.2014 13:00:00
01.12.2014 13:00:00
Reload of ManagementDashboard.qvw
Waiting
26.11.2014 13:00:00
27.11.2014 13:00:00
Reload of MESC_Loader.qvw
Waiting
18.11.2014 13:36:36
On EDX
Reload of MESC_Main.qvw
Waiting
18.11.2014 13:35:25
On EDX
Reload of MESC_Online.qvw
Waiting
18.11.2014 13:35:55
On EDX
Last updated @ 26.11.2014 18:41:00
Automatic refresh of task list
Refresh

# 2 The Repository

## 2.1 Overview

The data of the repository is used in multiple ways:

- The repository defines and describes the **interface between client and server**. The input parameters and the result sets of service requests are described.
- In case of interpreted service types, the **processing and the business logic of a service** is specified via configuration in the repository. Only in exceptional cases, an actual programming in the server is required.
- For the client, the repository defines **how the data is displayed on the client** and which GUI elements are used to enter data. The repository also defines how the client checks the user input. You can generate most of the applications on the client using the configurations of the repository. Here, programming on the client is not required.

The repository data is grouped and structured using **domains**. A domain summarizes all data that logically belongs to an application.

The domain contains hierarchically structured and typed data. A domain includes *services* and *service parameters*, the respective *GUI settings*, *properties*, *authorizations*, *ReferenceData* and *ControlDataSources*.

Find below a detailed description of the repository elements.

## 2.2 Domain

Domains have properties and provide services within the domain context.

A domain is the smallest software unit. You can update the domain using an update package. Create a separate domain for each application. This domain then includes the services implemented for this application. You can also use the services and client attributes of a domain in applications of other domains. For example, a client application in its own domain can use a service of a different domain.

You can assign global contents to a global domain: for example, client menu configurations or separate global syntactic types.

### Name

Each domain has a unique name. For the name, you use the notation "UpperCamelCase".

## 2.3 Service

Services have transfer parameters and return values, which are often identical to the properties of the domains.

### 2.3.1 Name

Name of a service. The service name usually consists of the domain name that includes the service and the **function**, separated by a dot.

### 2.3.2 Function

This field describes the requested service function. Typical functions are list, update, insert, delete, new, ...

### 2.3.3 ServiceType

There are several service types.

***InterpretedJavaService2***: Services of this type are used to display lists and evaluations. The services are interpreted at runtime using repository data. Contrary to the *InterpretedJavaService,* the services of type *InterpretedJavaService2* are prepared to stream data and provide more elegant options for Java user exits.

***InterpretedJavaService*** *(obsolete)*: Services of this type are interpreted at runtime using repository data. These services have been replaced with the service type *InterpretedJavaService2.*

***InterpretedBAPIService***: You use services of this type to edit data. The services are interpreted at runtime using repository data.

***ExternalJavaService***: Services of this type are completely implemented in Java. You can use these services to implement lists or editing functions. You use these services if the possibilities of the interpreting service types are not sufficient and the logic must be converted into Java programming.

***InterpretedWrapper***: Services of this type are interpreted at runtime using the repository data. The service is implemented as wrapper of an existing PDM dialog and is therefore subject to specific limitations, e.g. it does not support any dynamic Where.

***Wrapper*** *(obsolete)*: Services of this type are programmed and wrap an existing BAPI function. They are therefore subject to specific limitations, e.g. no dynamic Where.

***JavaService (obsolete)***: Services of this type are completely implemented in Java.

Recommendation:

- The type *InterpretedJavaService2* is recommended for services that you use to read data.
- The type *InterpretedBAPIService* is recommended for services that you use to write data.
- If the interpreted service types cannot meet the requirements (or only with great effort) even if they include Java user exits, you should use the services implemented in Java of type *ExternalJavaService*.
- The other service types are older technologies and should not be used for new developments.

### 2.3.4 ListMode

For services of type *Wrapper* or *InterpretedWrapper*: This column must be populated for each service. The column specifies whether the requested PDM dialog returns a file as result or whether it is only a return string. "Y" => The result is a file, otherwise only a string.

### 2.3.5 DLG

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper,* you must fill in either **DLG** or **SystemCall.** You fill in DLG, if **ServiceType** is *Wrapper* or *InterpretedWrapper* and if the service requests a PDM dialog with the structure "DLG=<content in this column>|..."

### 2.3.6 SystemCall

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall.** Fill in SystemCall, if the you want to run a program in the server. In the column, the name of the external program is specified. The result is a PDM dialog with the structure: "DLG=SYSTEM.CALL|PROG=<content of this column>|...".

## 2.4 ServiceGui

The ServiceGui data define the use and the presentation of the services on a client. You can clearly allocate the ServiceGui to a service via their name.

### 2.4.1 Name

The name of the service for which this data record provides presentation information.

### 2.4.2 Package

This field is obsolete and must be left empty.

### 2.4.3 Extended

This field is obsolete and must be left empty.

### 2.4.4 AdditionalDataLogics

This field is obsolete and must be left empty.

### 2.4.5 ApplicationID

Application ID used for generating applications in the client. In case of editing applications, the ApplicationID is edited with the main data source of the application that you want to generate.

### 2.4.6 ApplicationTitle

Language key for the title of the generated application. In case of editing applications, the ApplicationTitle is edited with the main data source of the application that you want to generate.

### 2.4.7 ApplicationHelpFile

File name of help file (including file extension) of the generated applications. In case of editing applications, the ApplicationHelpFile is edited with the main data source of the application that you want to generate.

The name of the help file should be independent of the technology of a used client. The client should therefore put a prefix in front of the file name. You can then design the help file displayed according to the client's technology.

Example for the client MOC: In ApplicationHelpFile, you enter "Article.pdf". The client MOC then loads the document "MOC_Article.pdf" as online help. The client automatically uses the prefix "MOC_".

### 2.4.8 ApplicationHelpIndex

Bookmark that is activated when Help is opened. In the main application, it is usually "Overview". You must only edit this bookmark for the main data source of the application that you want to generate.

### 2.4.9 Description

#### 2.4.9.1 General

Language key for short description of service.

You can show this description on the client when the selection of services is displayed.

#### 2.4.9.2 Processing in the MOC client

The MOC shows the description if you add a data source while configuring an application.

## 2.5 ServiceParameter

ServiceParameters specify the parameters of a service. They provide information on the data source and value ranges.

The service parameters include selection criteria and the columns of the result set. A service parameter can be a selection criterion or be included in the result set. The attributes described below specify if a service parameter is used as selection criterion and/or is included in the result set.

### 2.5.1 Acronym

Name of the parameter. The combination of **Acronym** and **ResultSet** must be unique for each service.

### 2.5.2 ResultSet

If the associated service returns more than one **ResultSet**, a name must be indicated here. This way, you can return results in parallel that have been calculated at the same time but have a different structure. The combination of **Acronym** and **ResultSet** must be unique for each service.

### 2.5.3 WebServiceType

Data type of the parameter *(decimal, integer, string, boolean, binary, datetime).* This value must be identical to the configured value of the property configuration. **IMPORTANT:** *binary* parameters are not supported by default. You can only use these parameters in user exits.

### 2.5.4 DefaultValue

Specifies a service default value for a parameter.

### 2.5.5 IsResult

Specifies whether this service parameter is part of the ResultSet (return value). If you want to use the **DefaultValue**, do not set this field (IsResult).

In case of services ot type InterpretedWrapper, you must only set the column IsResult to "Y" for UPDATE, LOCK, UNLOCK, DELETE, INSERT and COPY, if the BAPI actually returns a value, e.g. a new internal_id when you create new data records.

### 2.5.6 IsDynamicResult

Required for the generation of the Java function (for dynamic ResultSets, the column number must automatically be extended to the fixed number). Missing columns are added as empty columns (i.e. these columns are not computed).

### 2.5.7 InputAsArray

The client must transfer values in form of an array. **InputAsArray** is only reasonable in case of a quantity input parameter, i.e. if at least one of the two columns, **IsSpecialParameter** and **IsFilterParameter**, is set and a quantity operator such as BETWEEN or IN is possible.

Specify if a field is an array or not (with filters always *yes* except for Boolean type).
If true and no array or empty, then exception. Is currently only verified in case of mandatory special parameters.

### 2.5.8 IsSpecialParameter

Specifies whether or not the parameter is a special type controlling the service functionality (i.e. is not a filter parameter). For the **ServiceType** *Wrapper*, this is the only possible parameter type. In case of the **ServiceType** *JavaService*, it represents a special parameter not directly included in the WHERE condition but with different "controlling" effects. If you want to use the *Default Value* on the server side, do not set this field. In addition to the defined special parameters of standard processing, you can also use other special parameters in user exits.

### 2.5.9 IsFilterParameter

Specifies whether it is a filter parameter. If you want to use the **DefaultValue** on the server side, do not set this field.

### 2.5.10 IsMandatory

Specifies whether it is a mandatory parameter for the service. If true and parameter is missing, an exception is thrown. Is currently only checked for special parameters.

### 2.5.11 Can* (filter) operators

This option specifies whether the service supports the relevant filter operator for this parameter. Set the "Can*" fields for filter parameters.

Available operators:

- **CanEqual**
- **CanLike**
- **CanBetween**
- **CanIn**
- **CanNotEqual**
- **CanLt (Can Less Than)**

- **CanLte (Can Less Than or Equal To)**
- **CanGt (Can Greater Than)**
- **CanGte (Can Greater Than or Equal To)**

For technical reasons, each operator has a second operator that you should select is a data record must be selected, if the operator is applicable or if the comparative value is NULL. The operator **CanEqual** will only return a data record in case of equal values, **CanEqualOrNull** in case of equal values or if the data record value is NULL. Accordingly, there are the following operators:

- **CanEqualOrNull**
- **CanLikeOrNull**
- **CanBetweenOrNull**
- **CanInOrNull**
- **CanNotEqualOrNull**
- **CanLtOrNull**
- **CanLteOrNull**
- **CanGtOrNull**
- **CanGteOrNull**

Especially with List Services you should make sure that generally all parameters support all operators in order to achieve the highest possible selectivity. In general, the framework supports this for Java services.

- You may only set **CanIn**, **CanBetween**, **CanBetweenOrNull** and **CanInOrNull,** if **InputAsArray** is also set.
- **CanLike** is only useful if the **WebServiceType** is *string*.
- With WebServiceType boolean, only **CanEqual** is useful.
- With WebServiceType string, all operators are possible.
- With all other types, all operators except for **CanLike** and **CanLikeOrNull** are useful.

Before you set wrappers, you must check which operators are actually supported by the PDM dialog or the system command.

## 2.5.12 HydraAcronym

With service type *InterpretedWrapper,* the HYDRA acronym is specified.

## 2.5.13 HydraResultAcronym

If the acronym of the selection criterion is different to the acronym in the result file, you can enter an acronym that is different to the HydraAcronym for the service type InterpretedWrapper and ListMode=Y.

### 2.5.14 TransferEmptyValuesToHydra

Specifies whether blank values, too, are to be transferred to the server, or whether the ID is simply omitted. "Y" => blank values are transferred, otherwise => ID is completely omitted.

Note: You must set this field for Insert and Update (editing screens). Only then, you can enter blank values and/or overwrite existing values with blank values.

### 2.5.15 HydraShiftPart

The following components are combined with the **Reference** field: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. These components are marked as belonging together. The column "HydraShiftPart" can include the following values:

- beginDate
- beginTime
- beginDatetime
- endTime
- endDatetime

Important: The column can only be populated if the parameter is part of a group that includes the following five components: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. The column must not be populated if it is only a group of three components including date, time and date + time field. In this case, ONLY populate the **Reference** column.

### 2.5.16 Reference

Is used to generate a DateTime data type from one field each for the date and the time (in seconds after midnight) and to identify the shift parameters.

### 2.5.17 TransformationType

Use this field to specify transformations for input and result parameters for List Services/wrappers (e.g. convert Bool to J/N and vice-versa or correct filtering for DateTime fields that consist of two fields in the database). For further details on this field, refer to section 2.10.

### 2.5.18 PlugName

Specifies whether the result parameter for this service is directly derived from the specified DataObject or whether it is added to the DataObject via plug.

Example:
Service A.List uses a plug of service B.List in the service parameter b. Consequently, the following configuration applies to service A.List:

| ServiceParameter | DataObjectName | PlugName |
|---|---|---|
| **a** | A.List | |
| **b** | A.List | B.List |

If the field *PlugName* includes a value, the Interpreter replaces the values of the ServiceParameter with those values of the plugged service when creating the SQL statement.

In the special case where an interpreted List Service does not use an own table but only plugs, and subsequently adds fields via user exit, these fields should state USEREXIT!

If you create new services, it is recommended to avoid plugs and to provide data directly via the DataObject via Join. Dependencies between several services are thus avoided.

## 2.5.19 DBField

Database field that you use to make a selection. Write the database field in lower case. You can either enter simply the field name or (for complex expressions) the expression with placeholders for the alias (e.g. hydadm.get_datetime(%1$s.bearb_date,%1$s.bearb_time) or {fn substring(%1$s.field,2,1)}).

Proceed as follows for joins to other tables:

Entry: <ALIAS>.<DBfield>

Example:

DB field: STA1.status_bez

Acronym: gage.status.designation

Table: caq_status (STA1)

Conditions: status_typ = ‘PMSTATUS’, status_nr = status

## 2.5.20 DBAlias

The alias for the table that is used to select the value for the acronym.

## 2.5.21 DBTabelle

The table that is used to select the value for the acronym.

## 2.5.22 DBFieldAlternative

If you cannot use the **DBField** because the **ConditionalFieldKey** is not applicable, you use the **DBFieldAlternative**.

You can enter a number, "null, 'string', {fn ...} or another field / subselect. If it is another field or subselect, you MUST enter %1$s for the alias of the table.

If **DBFieldAlternative** is empty, but you require an alternative field, NULL is selected.

## 2.5.23 DataObjectName

If a service uses several data sources to identify its data, you can store the data source (= DataObject = DO) that issues the result parameter in this field. For example: A service includes the parameters a, b and c:
- a is computed,
- b is identified using data object (DO) F and
- c is identified using data object (DO) G.

For a: the field is blank. For b: the field contains F. For c: the field contains G. Is used as reference for the ...do.xml configuration.

## 2.5.24 ConditionalFieldKey

This field specifies if a DB field is only conditionally available. The ConfigurationManager checks the condition for the existence of the field. Enter the feature key of the Configuration Manager **(feature set)** in this repository field to enable the check.

If a parameter is a conditional field and the condition is not fulfilled, the entries for the MOC acronym are removed from the ComplexSelectMap and the SpecialFilterMap.

As a result, the changes in the Special Filter Map via user exits and transformation type are also lost!

### 2.5.25 Constraints

Constraints are processing parameters that are used for **ServiceType** *InterpretedBAPIService*. Constraints are structured as keys with optional values. The separator between keys is the pipe character (|). You use a semicolon to separate various values. You use the equal sign (=) to separate key and value. The general structure is as follows:

Key1=Value;Value;Value|Key2|Key3=Value|

The following constraints are available:

| Constraint Key | Constraint value(s) | Description |
|---|---|---|
| KEY | exactly one number between 1 and 5 | Define field as key including key number for hyd_lock table |
| SERIAL | none | Field is a SERIAL (and/or auto-increment) |
| SEP_DATETIME | 1st parameter refers to the date field<br>2nd parameter refers to the time field | Allows processing of separate date and time fields |
| BOOL | 1st parameter is the value to be entered into the DB if true<br>2nd parameter is the value to be entered into the DB if false<br>3rd parameter is the value to be entered into the DB if null (null for null)<br>4th parameter is the type of DB field, e.g.<br>BOOL=J;N;null;string\|<br>BOOL=1;0;null;integer\| | Use this to write Boolean values into a string or Integer Field. |
| MODIFY_TS | None | |
| MODIFY_BY | None | |
| CREATE_TS | None | |
| CREATE_BY | None | |

## 2.6 ServiceParameterGui

The ServiceParameterGui define how ServiceParameters are displayed on the client. Use **Acronym** and **ResultSet** to clearly allocate ServiceParameterGui to a service parameter.

A number of settings exist in the ServiceParameterGui and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the ServiceParameterGui if you want to display specific services on the client in a way that is different to the settings in the properties. The ServiceParameterGui fields overwrite the property fields of the same name.

### 2.6.1 Acronym

Name of the parameter for which this data record provides presentation information. There must be a corresponding property for each **acronym** of a parameter.

### 2.6.2 ResultSet

See **ResultSet** with ServiceParameter.

### 2.6.3 Label

#### 2.6.3.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

#### 2.6.3.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

### 2.6.4 Tooltip

Specifies a specific tooltip for the parameter in the service context. Entry as language key.

### 2.6.5 FormatType

Use this field to overwrite specific values of a property in relation to the service (currently **Label, Length**, **ControlType**, **ControlTypeMode**, **ControlDataSource**, **ControlDataSourceMode**, **ControlDataSourceResult**).

For example: If you enter *workplace.id* as **FormatType** for the parameter *resource.id*, you can define for the parameter to be a *resource.id* in this service, however its length, label and control properties are taken from *workplace.id*.

In this case (other than in case of semantic and syntactic types), the value from **FormatType** takes priority. For this reason, we have a new hierarchy:

- Value from **FormatType**
- Value from ServiceParameterGUI
- Value from Property
- Value from **SemanticType**
- Value from **SyntacticType**

## 2.6.6 ClientDefaultValue

Input fields have a **ClientDefaultValue** property. The value entered here is displayed as default value when the control is initialized. "From" and "to" values are separated by semicolons.

**Set checkbox:** If the value of this field is set to *true* during a CheckEdit, the checkbox is set after initializing.

**Preallocation of text fields with "from" and "to" values (InputAsArray):** set value1;value2 to prepopulate the 'from' and 'to' fields during a text edit.

**Date fields:** In case of date fields, the field can be preallocated with an offset. If you set default values for date fields, you must absolutely specify the type of offset. The following offsets are possible:

- h (hours)
- d (days)
- w (weeks)
- m (months)
- y (years)

The 'to' value is always **relative to the 'from' value.** The default value is always a DateTime object. The presentation depends on the output format of the relevant field.

You can put "[" and "]" in front and at the end of the relevant value to specify the start and end of a period of time. Consequently, e.g. "[0d;0d]" means that 12:00:00 AM is entered in the 'from' field today and 11:59:59 PM is entered in the 'to' field today. "[-1w;0w]" means from Monday last week up to Sunday last week.

**Examples**

Current date:

```
0d
```

From today to the day after tomorrow:

```
0d;2d
```

From today to one week from today:

```
0d;1w
```

From yesterday to tomorrow:

```
-1d;2d
```

From one year ago today to one year from today:

```
-1y;2y
```

**Year shortlists:** You can configure a year shortlist by **ControlDataSource** = YearList and **ControlDataSourceMode** = Script, or even by standard "Service-ControlDataSource". In this case, you can use the following default values:

- Current year: 0y and/or currentyear
- Last year: -1y
- Following year: 1y
- 4 years ago: -4y
- Year that was current 10 months ago: y-10m → this is mostly the case when the relevant year field is used in combination with a month shortlist.

If you want to preallocate two fields **(ShowSecondControl)**, you have to separate both values by semicolon, e.g. -1y;1y

**Month shortlists:** You can use the following default values for a month shortlist:

- Current month: 0m
- Last month: -1m
- Following month: 1m
- 4 months ago: -4m

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y.

### 2.6.7 IsKey

The IsKey column is very important and should be occupied for all key columns of a service, since otherwise data records cannot be clearly identified. Columns including the value 'null' may NOT be defined as keys. The IsKey columns should be identical for all services (insert, update, delete, lock, unlock, copy). These entries are important, so it is best to verify them twice.

This field specifies the positioning of the cursor after an editing operation. If the positioning option OnKeyValue is selected, the client should only request one new data row after editing. You also use values that are marked **IsKey** as selection criteria. **IsKey** must also be set for *delete,* since this data record must be deleted from the view.

**IsKey** must also be indicated for *list.* If no sorting is given in the *list,* sorting takes place according to **IsKey** fields.

Every parameter which is **IsKey** MUST always be **IsMandatory**. This rule has two exceptions:

- List service
- Wrappers with composed keys.

### 2.6.8 ShowInGrid

Specifies whether the parameter is to be displayed in tables by default.

### 2.6.9 ShowInDetail

Specifies whether the parameter is to be displayed in detail views by default.

### 2.6.10 ShowInSearch

Specifies if the parameter is to be used as selection criterion (i.e. in selection panels) by default.

### 2.6.11 ColumnCategory

#### 2.6.11.1 General

In the tabular view, the client should provide the option to summarize the columns in the table to categories. You specify a language key that is displayed as title of the summarized columns.

#### 2.6.11.2 Processing in the MOC client

The ColumnCategory is used to assign the parameter to a "strip" in the grid (table view).

## 2.6.12 Category1, Category2, Category3

### 2.6.12.1 General

The client processes the columns Category1, Category2, Category3 in order to group fields in applications. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

### 2.6.12.2 Processing in the MOC client

Category1: Assigns the parameter to a tab in the detail view.

Category2: Grouping options for detail screens.

Category3: Currently not used.

## 2.6.13 TabOrder

You specify the order of tabs for detail views.

## 2.6.14 ColumnOrder

You specify the order of columns in tabular views.

## 2.6.15 ShowSecondControlInSearch

### 2.6.15.1 General

Specifies whether a second control is to be displayed (from/t0). You can use this setting with selection criteria that include a value range via the operator CanBetween, e.g. "date from/to".

### 2.6.15.2 Processing in the MOC client

The MOC provides two adjoining fields. The label text of the second field is automatically "to". If it is a field of "date" type, you can predefine a relative date for both fields.

## 2.6.16 SearchTabOrder

Specifies the tab sequence for the selection panel.

## 2.6.17 SearchCategory1, SearchCategory2

### 2.6.17.1 General

The client processes the columns SearchCategory1 and SearchCategory2 in order to group fields in selection panels. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

### 2.6.17.2 Processing in the MOC client

SearchCategory1: You allocate the parameter to a tab in the selection panel.

SearchCategory2: Grouping options for the selection panel.

## 2.6.18 ControlType

Use the ControlType to specify which control should be used for the relevant parameter. The client will map the abstract type onto a specific control class. If you do not specify a type, the client uses the data type to decide on the ControlType. Possible values for the ControlType:

**CheckEdit:** Selects a Boolean value (true/false) or multiple values if a reference to a data source is given.

**ColorEdit:** Selects a color value.

**ComboBoxEdit:** Combobox with selection of values from web service or data reference.

**DateTimeEdit:** Enter a date and/or a time.

**MemoEdit:** Enter an arbitrary text.

**RadioGroup:** Selects a Boolean value (true/false) or one of multiple values if a reference to a data source is given.

**TextEdit**: Standard text input. You can add a button opening a search dialog to this control, if you add a reference to a service in **ControlDataSource**. If you enter the name of a DataLogic in **ControlParameter** and if a mapping is included in **ControlDataSourceResult**, data will be requested upon leaving the control and return values will be mapped appropriately.

## 2.6.19 ControlTypeMode

### 2.6.19.1 General

Allows for controlling the input control.

**CheckEdit:** DualState (default), TriState, J;N;J (checked;unchecked;tristate)

**ColorEdit**: none

**ComboBoxEdit**: SingleEdit, Single, Multiple (multiple selection)

**DateTimeEdit**: Date (date display), Time (time display), DateTime, RelativeDate, RelativeDateTime

**MemoEdit:** none .

**RadioGroup:** SingleColumn, SingleRow

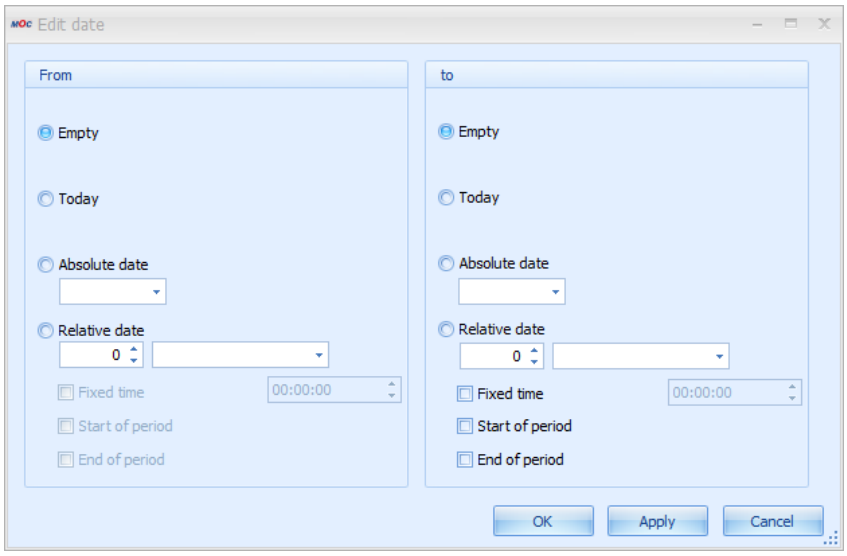
**TextEdit:**

- Empty: the search button is shown if a **ControlDataSource** is defined.
- "SearchButton": Search button is shown.
- "SearchButtonValidate": Search button is shown. If you enter an invalid value, an error is displayed.
- "OpenFileDialog": opens a file selection dialog.

## 2.6.19.2 Processing in the MOC client

If you use **DateTimeEdit** including the definition of a relative date **(ControlTypeMode**: RelativeDate or RelativeDateTime), you can enter a relative date.

If **ShowSecondControl** = true, you can predefine the complete relative value range. In this case, a button is displayed behind the second input control. You can use this button to open the following dialog:




Use this dialog to customize the values for **ClientDefaultValue** . The following entries are possible:

- **Empty:** no value is adopted

- **Today:** the current date is adopted
- **Absolute date: you can select** a fixed date value via a calendar control
- **Relative date: you can select and adopt** a date relative to the current date. In this context, "Start of period" means that you additionally go to the start of the selected period. Example: current date is 20-MAY-2010. If you select "- 1 month", 20-APR-2010 is adopted. If you also select "Start of period", the date is changed to 01-APR-2010. The same applies to "End of period". These settings are saved in the mpdvEdit or the selection profiles as **ClientDefaultValue.**

## 2.6.20 ControlParameter

See **ControlType** → TextEdit

## 2.6.21 ControlDataSource

Data source for the selection of values. The data source can be:

- **Web service** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = **Name** of a ControlDataSource. See also section "2.8 ControlDataSource")
- **ReferenceData (**configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Reference*, **ControlDataSource** = **Type** of ReferenceData)
- **Search application** (configuration: **ControlType** = *TextEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = application name)
- **Script** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Script*, **ControlDataSource** = Name of script)

## 2.6.22 ControlDataSourceMode

Data source mode (*Lookup, Reference* or *Script*).

## 2.6.23 ControlDataSourceParameter

Optional setting of parameters of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

See also the description **ControlDataSource - Parameter**

## 2.6.24 ControlDataSourceResult

Optional setting of the result of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

The settings in this field provide more options than the **Result** in the **ControlDataSource:**

Result columns are separated by semicolon. Field mapping:

- First entry: Value
- Second entry: Labeling
- Third entry: UnitLabel
- As of the fourth entry, the fields are mapped:
    - Via acronym or semantic type.
    - Field mapping: in **ControlDataSourceResult**, you can enter a mapping in the form "FieldName=ColumnFromResult" as from the fourth entry. For example, you can specify tool.id=resource.id in order to fill the field "tools.id" with the "resource.id" value from the search application. Several mappings are separated by ";" - spaces are not allowed.
    - Asterisk mapping: Instead of mapping, you can also enter * . Subsequently, all return columns of the search application are mapped. The mapping is performed as usual via ID or semantic type.

## 2.6.25 VisibleCondition

This value decides whether an input field is visible on the client. For customization, see EditableCondition.

## 2.6.26 EditableCondition

This value decides whether you can edit an input field on the client. There are three possibilities:

- Boolean value: In case of *TRUE* or *FALSE,* the field is always editable / non-editable.
- Binary expression:
    - Field name must be the name of a field that is also located in the ControlPanel.
    - Valid operators: =, <, >, <=, >=, <>, !=
    - The value is written as a string and interpreted depending on the comparative field value.
    - Field, operator and value must be separated by a space!
- Concatenation of binary expressions:
    - You can concatenate an arbitrary number of binary expressions.
    - You can use the operators "&&", "AND", "||", "OR" to link expressions.
    - Here, too, all components of the conditions must be separated by a space.
    - Priority of operators: "AND" or "&&" are evaluated first, then "OR" and "||".
      You cannot use brackets.
    - Example: resource.id = 12345 && resource.costcenter = 20 || resource.id = 60610

The client assigns the default value of the property "ClientDefaultValue" to the field, if the result of an expression in the EditableCondition or the VisibleCondition changes from FALSE to TRUE. The client dynamically evaluates the expressions in the EditableCondition and the VisibleCondition, if the fields of the application change.

## 2.6.27 ScriptId

### 2.6.27.1 General

The ID of the script that is allocated to the parameter. If you set the ID, the relevant script is performed upon various events (at present EditValueChanged and Leave).

### 2.6.27.2 Processing in the MOC client

The method name of the script is ScriptId+EditValueChanged and/or ScriptId+Leave. The script can be included in any DLL that is read by the CodeManager.

## 2.7 Property

For the acronyms, properties include information on data types, input and output formats, display options, a name (that can be localized) and other settings specifying how ServiceParameters are displayed in the client. Each property has a system-wide unique acronym.

A number of settings exist in the ServiceParameterGui and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the ServiceParameterGui if you want to display specific services on the client in a way that is different to the settings in the properties. The ServiceParameterGui fields overwrite the property fields of the same name.

## 2.7.1 Acronym

Clear identification of the property across all domains.

## 2.7.2 WebServiceType

Describes the data type used to transfer the property between client and server. The currently supported WebServiceTypes are exclusively

- *binary*
- *boolean*
- *datetime*
- *decimal*
- *integer*

- string

Important: the types **date* and **time* are internal types which are not transferred.

### 2.7.3 NETType

The data type used by the client. If NETType is empty, the **WebServiceType** is used to automatically identify the data type used by the client. At present, NETType supports the following entries:

- **color:** Use *color* to convert the transferred integer into an RGB code. In this case, the conversion is implemented by the grid.
- **duration:** creates a duration from an integer.
- **image:** either creates an image from a transferred byte array or interprets a transferred string as image name. For example, the *maintenance.active.led* property is transferred as a string including the name of an icon.
- **preview:** Specifies that the contents in the client may be displayed as "preview" (similar to auto-preview outlook) (application e.g. in DevExpress grid).
- **timestamp: Use *timestamp*** to automatically create an additional column for date values in the client in order to process time and date separately.

### 2.7.4 SemanticType

Use semantic types to inherit semantic properties. The "order.id" is therefore used to identify orders (semantic meaning). The acronym *operation.order.id* includes such an order identification and therefore has the semantic type *order.id.* If an attribute of the property is not set (empty), the respective value from the semantic type is used for the processing in the client.

For example: You must set the semantic type if you want to adopt a value from a lookup screen in the field. For the *workplace* field, enter e.g. resource.id as semantic type in order to adopt the selected workplace from a search screen for workplaces. Refer to the description of the SyntaticType for further information on the priority used to specify the attributes of a Property, the SemanticType and the SyntacticType.

### 2.7.5 SyntacticType

You mainly use a syntatic type for a uniform presentation of the different properties. The syntactic type does not include any semantic content. For example: The properties booking.begin_ts and booking.shift.start_ts have different semantic meanings, but are presented in a uniform format that can be controlled centrally.

Syntactic types are used to control the characteristics of a *Property:* for example length, input and output screen, tooltip, label, etc. To select the valid value for a characteristic, the client proceeds as follows:

- If the characteristic (e.g. length) is set in Property, the client uses this value.
- Or: If a semantic type is available and the characteristic is set, the client uses this value.
- Or: If a syntactic type is available and the characteristic is set, the client uses this value.

Note:

- You must always enter a **description** for syntactic and semantic types.
- Syntactic types can reference other syntactic types so that "inheritance hierarchies" can be created.
- Create syntactic types as property of the *SyntacticType* domain.
- Semantic types are usually "real" properties of a "normal" domain that are used as semantic type at other places.

## 2.7.6 Label

### 2.7.6.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

### 2.7.6.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

## 2.7.7 DefaultTooltip

Specifies the default tooltip for the property as language key.

## 2.7.8 UnitLabel

Text key for unit. The unit is displayed to the right of the input field.

## 2.7.9 OutputFormat

This field specifies the format that is used to display a value (e.g. for date or quantity values). If you do not enter an **InputFormat** in the repository, the MOC tries to develop an appropriate format from the **OutputFormat.** Enter the value **InputFormat** in the repository only if special masking is required. Find further details in section "2.7.12 Rules for the input/output formatting".

## 2.7.10 InputFormat

Equivalent to OutputFormat. You can enter a valid regular expression in the field InputFormat. Other entries that are not regular expressions are not permissible. Find further details in section 2.7.12.

## 2.7.11 Length

The client shows the control for this acronym in the specified width (i.e. the specified number of characters). With Length=0, the control uses the entire width available. If a width is specified but the space available is not sufficient, the control is cut off.

This field also specifies the number of characters that you can enter in an input field with **ControlType = TextEdit,** if no other **InputFormat** is specified.

## 2.7.12 Rules for the input/output formatting

### Overview

In the repository, you define the formatting of the data output and the input dialogs to edit data. The "Properties" of the different acronyms include an **OutputFormat** and an **InputFormat.** The **OutputFormat** defines formatting if you display a value.

**Important:** If you do not enter an **InputFormat** in the repository, the MOC uses the **OutputFormat** to generate an appropriate formatting. Enter the value InputFormat in the repository only if special masking is required.

In case of strings, you cannot enter the special characters asterisk (*) and pipe (|), if you have not defined any input format. As you use these two special characters as separator and control character, they can cause problems if they are written in the database.

With strings, the maximum number of characters that you can enter is defined by the attribute **Length**, if no other input format is defined.

**Syntactic types**

The Properties provide so-called "syntactic types" in order to make groups (similar to field types in Delphi). Syntactic types have the same properties as real properties. The real properties have a syntactic type. For example, if the **output format** of the syntactic type includes a value, this value is used wherever this syntactic type is entered.

**Example: Industrial minutes**

The syntactic type "Durations" has the format {0:mpdv_timespan}. With the different properties showing durations, "Durations" is entered in the column **SyntacticType** and no entries are made in the columns "output format" and "input format". When the property is read - and if no **output format** is available in the property - the format of the syntactic type is used.

If a system displays industrial minutes (no standard function!) and if the syntactic type "Durations" is specified, the **output format** is automatically changed from {0:mpdv_timespan} to {0:mpdv_industrialMinutes}. As a result, all formats including the syntactic type "Durations" are shown in industrial time units.

Times and durations are internally stored in the system as integer seconds. If you convert times or durations during input or output formatting to formats other than hours, minutes and seconds (HH:MM:SS), the conversion may not be possible without losses. For example, this applies to the use of the "mpdv_calc" format and the classic industry minute display:

When converting from seconds to hours (division by 3600), decimal numbers with an infinite number of decimal places can occur, which inevitably have to be rounded when displayed on the client. Example: 20 minutes = 1200 seconds = 0.333333… hours. If the value is rounded to three decimal places, you calculate backward as follows: 0.333 * 3600 = 1198.8 seconds. Depending on how the client rounds, the internal value is then no longer 1200 seconds, but 1999 or 1998 seconds.

If you use less than three decimal places, the conversion error gets even greater:

The system recorded a duration of 123 seconds. The client displays 0.03 hours. If you calculate backwards, the result is 108 seconds.

## Output formats

| OutputFormat | Automatically created masking (input format) | Examples | Description |
|---|---|---|---|
| **Numeric data** | | | |
| f(number) | None | f3, f1 | Numeric value without thousands separator. The number specifies the number of decimal places. |
| n(number) | None | n0, n2, n5 | Numeric value with thousands separator. The number specifies the number of decimal places, even if the data type to be displayed is an integer type. In case of n0, no decimal places will be shown. |
| #.(##) ,<br>#.(0) | None | #.####,<br>#.0000 | Arbitrary format |
| {0:mpdv_timespan} | None | 2:33:30 | MPDV format provider. Conversion of seconds to hh:mm:ss and vice-versa. |

| | | | |
|---|---|---|---|
| {0:mpdv_timespan_short} | None | 2:33 | MPDV format provider. Conversion of seconds to hh:mm and vice-versa. |
| {0:mpdv_timespan_minutes} | None | 45 | MPDV format provider. Conversion of seconds to minutes and vice-versa. |
| {0:mpdv_cycletime} | None | 1:30:00 | MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa. |
| {0:mpdv_te} | None | 2.00 | MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa. |
| **Strings** | | | |
| empty | [^*\|]]* | | Illegal characters begin with **^.** In this example * and \| * **No limitation in length** |
| empty | [^*\|]{0.10} | | Illegal characters begin with **^.** Max. length: 10 characters |
| empty | [0-9a-fA-F] | | Allowed characters 0 through 9, a through f, A through F. |
| **Special formats** | | | |
| {0:mpdv_cycletime_sec_cycle} | None | 29 sec/cycle | MPDV format provider. Seconds per cycle. Conversion into seconds per 1000. |
| {0:mpdv_IndustrialMinutes} | None | 1.50 | MPDV format provider. Conversion of seconds into industrial minutes and vice versa. |
| {0:mpdv_leadingzeros_order} | ORDER | | You must combine this output format with the input format ORDER. The combination is used in the syntactic type "order_id". The basic settings are used to automatically specify the length. |
| {0:mpdv_leadingzeros_operation} | ORDER | | You must combine this output format with the input format ORDER. The combination is used in the syntactic type "operation". The basic settings are used to automatically specify the length. |
| {0:mpdv_leadingzeros_sequence} | ORDER | | You must combine this output format with the input format ORDER. The combination is used in the syntactic type "ordersequence_id". The basic settings are used to automatically specify the length. |

## Input formats

The following definitions are available for the input format:

- Leave empty: The input format is implicitly defined using the output format. See table above.
- Use of logical input formats
- Use of regular expressions

## Logical input formats

To simplify the definition of input formats and limit the variety of entries in the repository, the logical input formats are provided. These input formats are permanently implemented in the client and can directly be used in the repository. Input formats are customized in the properties. But service parameters specify whether wildcards are allowed. For this reason, the input format actually used can vary depending on the allocated service.

In order to use logical input formats, define the name of the input format in the affected property in the repository. The following formats are currently available:

| Name | Input format without wildcard | Input format with wildcard |
|---|---|---|
| **CHARACTER** | [^*][LENGTH] | [^|][LENGTH] |
| **NUMBER_N0** | [0-9][LENGTH] | [0-9][LENGTH] |
| **NUMBER_N1** | [0-9][LENGTH]\R.?[0-9]{0,1} | [0-9][LENGTH]\R.?[0-9]{0,1} |
| **NUMBER_N2** | [0-9][LENGTH]\R.?[0-9]{0,2} | [0-9][LENGTH]\R.?[0-9]{0,2} |
| **NUMBER_N3** | [0-9][LENGTH]\R.?[0-9]{0,3} | [0-9][LENGTH]\R.?[0-9]{0,3} |
| **NUMBER_N6** | [0-9][LENGTH]\R.?[0-9]{0,6} | [0-9][LENGTH]\R.?[0-9]{0,6} |
| **TIMESPAN_SHORT** | [0-9][LENGTH]\R:[0-9]{2,2} | [0-9][LENGTH]\R:[0-9]{2,2} |
| **TIMESPAN** | [0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2} | [0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2} |
| **ORDER** | [0-9a-zA-Z.+][LENGTH] | [0-9a-zA-Z.+*][LENGTH] |

The placeholder [LENGTH] is replaced with the configured field length at runtime. If the defined length is '0', an '*' is entered. With the logical format "ORDER", the system automatically changes the [LENGTH] according to the basic settings when the output format changes.

## Input/Output formats including calculation

If you specify the **output format** *mpdv_calc,* you can include calculations in the formatting. In the format, you can specify a divisor and multiplier and an identifier that specifies if a reciprocal is calculated. You can also specify the number of decimal places. The **OutputFormat** *mpdv_calc* implicitly defines the **InputFormat**. If an input of values is made, the reciprocal value is calculated.

Example: "mpdv_calc;MULT=5;DIV=2;INVERSE=false;FORMAT=n3" (the value is multiplied by 5, divided by 2, then the reciprocal is calculated and the result is displayed with 3 decimal places).

The input/output format including calculation is normally used for the display of cycle times or specifications of single pieces. In the database, these times are always saved in seconds per 1000 pieces. If an input/output format including calculation is used, you can convert the times to hours per 1000 pieces, minutes per piece or with reciprocal also to piece per hour.

## Overview of regular expressions

You can find a large amount of information on regular expressions using the search engines on the internet. In the following, the most important aspects are presented.

**Meta characters**

Represent a range of characters.

| Character | Description |
|---|---|
| **.** | Matches any character. |
| **[aeiou]** | Matches any single character included in the specified set of characters. |
| **[^aeiou]** | Matches any single character, which is not included in the specified set of characters. |
| **[0-9a-fA-F]** | Use of a hyphen (–) allows specification of contiguous character ranges. |
| **\R.** | Matches the decimal separator specified by the System.Globalization.NumberFormatInfo.NumberDecimalSeparator property of the current culture. |
| **\R:** | Matches the time separator specified by the DateTimeFormatInfo.TimeSeparator property of the current culture. |

**Quantifier**

Repetition, number of characters

| Quantifier | Description | Samples |
|---|---|---|
| * | Specifies zero or more matches. | The "\w*" mask matches a string consisting of zero or more letter characters. It's equivalent to the "\w{0,}" mask. |
| + | Specifies one or more matches. | The "\w+" mask matches a string consisting of one or more letter characters. It's equivalent to the "\w{1,}" mask. |
| ? | Specifies zero or one match. | The "\w?" mask matches zero or one letter character. It's equivalent to the "\w{0,1}" mask.< /description> |
| ***{n}*** | Specifies exactly *n* matches. | The "\d{4}" mask matches exactly four digits. |
| ***{n,}*** | Specifies at least *n* matches. | The "\d{2,}" mask matches two or more digits. |
| ***{n,m}*** | Specifies at least n, but no more than m, matches. | The "\d{1,3}" mask matches either one, or two, or three digits. |

**Special characters**

Special characters

| Character | Description | Samples |
|---|---|---|
| **\|** | Alternation symbol. This can be used to implement a choice between two or more alternatives. | The "1\|2\|3" mask matches either "1" or "2" or "3". The "abc\|123" mask matches either "abc" or "123". The "\d{2}\|\p{L}{2}" mask matches either two digits or two letters. |
| **()** | Grouping. You can use parentheses to create sub-expressions, or to limit the scope of the alternation. | The "(an\|ba)t" mask matches either "ant" or "bat". The "(net)+" mask matches "net", "netnet", "netnetnet", ... strings. Compare with the "net+" mask which matches the "net", "nett", "nettt", ... strings. The "(0\|1)+" mask matches a string of indeterminate length, consisting of "0" and "1". |

**Examples**

Input 1..9999 => Input format for property : ([1-9]|[1-9][0-9]|[1-9][0-9][0-9]|[1-9][0-9][0-9][0-9])

Input 0..999 => Input format for property : ([0-9]|[1-9][0-9]|100)

### Best practice: input of long string fields

The client identifies the width of an input field using the attribute **Length**. In case of long string fields with more than 20 characters, the layout can become confusing because these string fields use the complete width of the layout and are very long compared to other input fields. Very long string fields are cut off on the right-hand side, if the available space is not enough. To avoid this behavior, you can control the displayed field width regardless of the number of characters that you can enter.

- Use the attribute **Length** to specify the **width** of the input field.
- You can use a regular expression in the **InputFormat** to specify the **number of characters that you can enter**.

If you enter strings that are larger than the displayed field, the input field automatically scrolls horizontally.

Examples:

| Attribute | Length | InputFormat | Effect |
|---|---|---|---|
| **article.designation** | 50 | .{0.250} | The field is displayed with a width of 50 characters. You can enter up to 250 characters. |
| **operation.input_component_list** | 25 | .{0.40} | The field is displayed with a width of 25 characters. You can enter up to 40 characters. |

## 2.7.13 FillChar

Obsolete. This field must be left empty.

## 2.7.14 Calculation

Obsolete. This field must be left empty.

## 2.7.15 Further fields see ServiceParameterGui

For a description of the following fields, refer to the data types of the ServiceParameterGui:

ControlType, ControlTypeMode, ControlParameter, ControlDataSource, ControlDataSourceMode, ControlDataSourceParameter, ControlDataSourceResult, VisibleCondition and EditableCondition.

## 2.8 ControlDataSource

A ControlDataSource defines a data source that you can use to fill selection lists in controls, for example. These can be data logics (service requests) or reference values (see also ReferenceData).

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents.

You use data logics to request services that identify selection lists (or RadioGroups) dynamically. For example, these lists can include master data that are configured in the database.

The settings made in the columns **Parameter** and **Result** can be overwritten in a **Property** or **ServiceParameterGui**.

### 2.8.1 Name

Name of the ControlDataSource. The name should be composed of English terms clearly describing the data source. You usually use the camelCase notation.

### 2.8.2 Source

If the data source is a web service, this field contains the name of the client's data logic. You derive the data logic from the service name. To do so, remove the dot between domain and function and use a capital letter for the first letter of the function:

| Service | Data logic |
|---|---|
| MDUser.list | MDUserList |
| MDUnits.list | MDUnitsList |

In case of reference values, this field includes the **Type** of a **ReferenceData**.

### 2.8.3 Parameter

A list of parameters. The list does not include spaces, use semicolons to separate parameters. This field is only allowed in combination with web service data sources. A parameter can be allocated dynamically or permanently.

Permanent parameters appear as <acronym>=<value>, e.g.

"dialogconfiguration.type=AIPDEF;dialogconfiguration.type=AIPTNR".

Dynamic parameters are specified as a pair of <acronym1>=**[**<acronym2>**].** e.g.

“resource.id=[resource.id];pdvprocessparameter.evaluation_ts=[pdvsinglevalue.evaluation_ts]”

The acronym in square brackets is replaced with the acronym values from the ControlPanel.

### 2.8.4 Columns

A list of requested columns. The list does not include spaces. To separate columns, semicolons are used. This is only permissible for web service data sources.

### 2.8.5 Result

You can enter 1-n acronyms separated by semicolon. The sequence used specifies the importance.

- Position 1 (Value): Name of acronym whose value is entered in the input field.
- Position 2 (ControlValue): Name of acronym whose value is displayed in the selection list. If you do not specify position 2, the acronym of position will be displayed.
- Position 3 (LabelValue): If you specify position 3, the value of the acronym is entered in the label field of the input field and also displayed in the selection list.
- Position 4-n: Use these positions to define additional return values, which are then used to update "dependent" controls in the client ("lookup").
  Only with web service data sources:
  Optional return columns of the data source, separated by semicolons. Without spaces. The return has the format <acronym>=<value> - for acronym pairs, the second acronym is therefore replaced with the result value (e.g. if you enter "operation.resource.id=resource.id", this results in "operation.resource.id=4711").

## 2.9 ReferenceData

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents. In contrast to values provided by web services, reference values are fixed and do not change. For this reason, reference values can be entered once in a list and are delivered in this form.

### 2.9.1 ref_data_key

The **ref_data_key** must be unambiguous for each entry. In special cases, this key is used in the source code (at least in the server).

Usually, the **ref_data_key** is composed of **type** + : + **db_key**; this facilitates its allocation to type and key. An exception occurs if the **db_key** includes a German expression. The **ref_data_key** must then be formed differently. For example, *pwdexclusion:person.firstname* is a super **ref_data_key** for the type *pwdexclusion.pwd* and **db_key** *PNR.PVORNAME*.

### 2.9.2 Type

Use this field to summarize various ReferenceData entries to a list.

### 2.9.3 db_key

The **db_key** is the actual value that is selected in the list. This key identifies an entry unambiguously within a **Type.** You cannot freely select the key because the key is often transferred to services and can correspond to the content of a configuration identifier in the database, for example.

### 2.9.4 is_default

The entry with this key is preallocated as default.

### 2.9.5 Designation

Text displayed in the selection list. A language key is specified.

### 2.9.6 sort_key

Specifies the sequence that is used to display the entries in the selection list.

## 2.10 Authorization

The authorization mechanism
- protects applications and functions against unauthorized use on the client,
- hides fields or field groups on the GUI,
- prevents these fields from being edited.

### 2.10.1 Authorization type

Controls the type of authorization. Possible values:

- Acronym: enables the authorization of individual fields (properties)
- AcronymGroups: enables the authorization to group fields
- Application: enables the authorization of applications
- Functions: enables the authorization of functions which are e.g. requested from the application toolbar.

### 2.10.2 Authorization Context

Context where the authorization is intended. If the field is left empty, authorization is always granted, irrespective of the context. You normally use this field to control the authorization of acronyms in the context of special services.

### 2.10.3 Authorization ID

Identifies the object to be authorized, i.e. the name of the acronym or the ID of an application.

### 2.10.4 Authorization key

The authorization key that is used to protect the object.

### 2.10.5 Authorization Designation

(Optional) text description of the authorization.

# 3 Repository Client

You use the MPDV Repository Client MRC to display and edit repository data. It provides a user-friendly access.

## 3.1 Quick start

This section provides a quick overview of how to work with the Repository Client. The individual steps are only briefly described. For further information on the individual steps, refer to the sections in the following, if required.

### Installation

Requirements

To use the Repository Client, you must have installed the Microsoft DotNet framework (at least version 4.5.2).

Program installation

To install the program, just copy the folder including the binary files into your system. An installation program is not required.

Installation of developer license

If the developer license is not available, you can only read the data. You cannot save or export the data.

The developer license is handed out once you have attended a respective Customizing Training. The developer license is provided as *.lic file. This file is included in the data medium that you have received during the training: Folder "Repository Client", subfolder "`tools/licence`", e.g. `x:\CUT-MOC_81_files\Tools\MPDVRepositoryClient\tools\licence\mpdvWrite.lic`.

Copy the folder "`licence`" with its content into the folder of the Repository Client in the roaming directory of the Windows user, e.g.:
`C:\Users\%User%\AppData\Roaming\MPDV\RepositoryClient\licence\mpdvWrite.lic`
This folder is automatically created on the first start of the Repository Client.

### Before you start

Before you start working with the Repository Client, you must make sure that the Repository Client has been installed according to the installation instructions and that the required license files are stored as described there.

In general, the repository is empty when you start work. However, if you do not want to start with an empty repository, it is recommended to make sure that the data you want to work with is available.

## First steps

Start the Repository Client.

➔ Start .\bin\mrc.exe

Now load or create a Workset.

A workset specifies the sources of the repository that you want to edit.

➔ Click the button *Load work set* in the file-based repository
-> select workingset.work

Click the button *Repository* → *Load Repository* to load the data from the sources specified in the workset.

How to proceed further depends on what you want to do via the Repository Client.

## Tip: Working with perspectives

The Repository Client provides the possibility to use different perspectives for different tasks. Do not hesitate to close views you do not need or drag them to another open view in order to tab them and hence provide for more space and clarity. You can also use more than one view of the same type. Simply adapt your perspective to the requirements of your tasks.

**Example:**
If you create a service and you would like to check with an existing service how to populate the fields, you can simply open another service view. Thus, you do not need to destroy your current view and re-orient yourself later.

**Default perspectives**

The installation of the Repository Client provides default perspectives:

**default**

This is a good perspective to start with. It provides a combined view for server and client-related contents.

Select a domain on the top left. Via the included relations, the top right area shows the services, servicesGUI and properties of this domain. The bottom right area shows the ServiceParameters of the service selected above, the ServiceParameterGui of the ServiceGui selected above and the ControlDataSources, ReferenceData and Authorizations of the selected domain.

**default client**

Similar to the "default" perspective but limited to the contents concerning client development.

**default server**

Similar to the "default" perspective but limited to the contents concerning server development.

**DB schema**

Shows documentation of the database structure.

**Validation**

Use this perspective to validate contents.

Proceed as follows:

- Open perspective and load data from workset.
- Perform validation: tab *Repository* --> button *Validate*.
- A CSV file listing the identified irregularities is generated in the sub-directory "validation_logs" of the Repository Client's installation directory. The application that is linked to this file type in the operating system opens the CSV file.
- In the views for Domains, Properties, ServiceParameters, etc. the Repository Client only shows the entries with detected irregularities.

You should analyze and, if necessary, correct the detected irregularities. Not every irregularity leads to an error.

## 3.2 Start and exit Repository Client

Start the Repository Client via the Windows start menu, a link on the desktop or the command line. As soon as all required components have been loaded, the application window is shown.

You can start and run the Repository Client multiple times in parallel on a PC. You can access different repository data in each of the started instances of the Repository Client. For example, you can view several versions of the repository at the same time.

You can also start the client by opening one of the workset files. On start of the client, the system attempts to load the contents of the workset defined in this file. This option is available after the first start of the Repository Client.

**Command line parameters**

You may transfer parameters to the Repository Client upon the start. The following parameters are supported:

`-perspective/-p <perspectivename>` Use this parameter to start the Repository Client with a specific perspective. If you do not use this parameter, the last active perspective is started by default.

`-workset/-w <worksetfile>` Use this parameter to specify the workset to be loaded. If you do not specify the workset on start of the application, the last loaded workset is loaded.

`--autoload/--a` If you use this parameter, the Repository Client loads the repository defined in the last active workset or in the workset transferred via parameters. This repository is loaded directly on start of the application.

`--trim/--t` If you use this parameter, the Repository Client removes so-called leading and trailing "whitespaces" when loading the repository. Only select this option if needed, because the load time increases extremely.

**Exit**

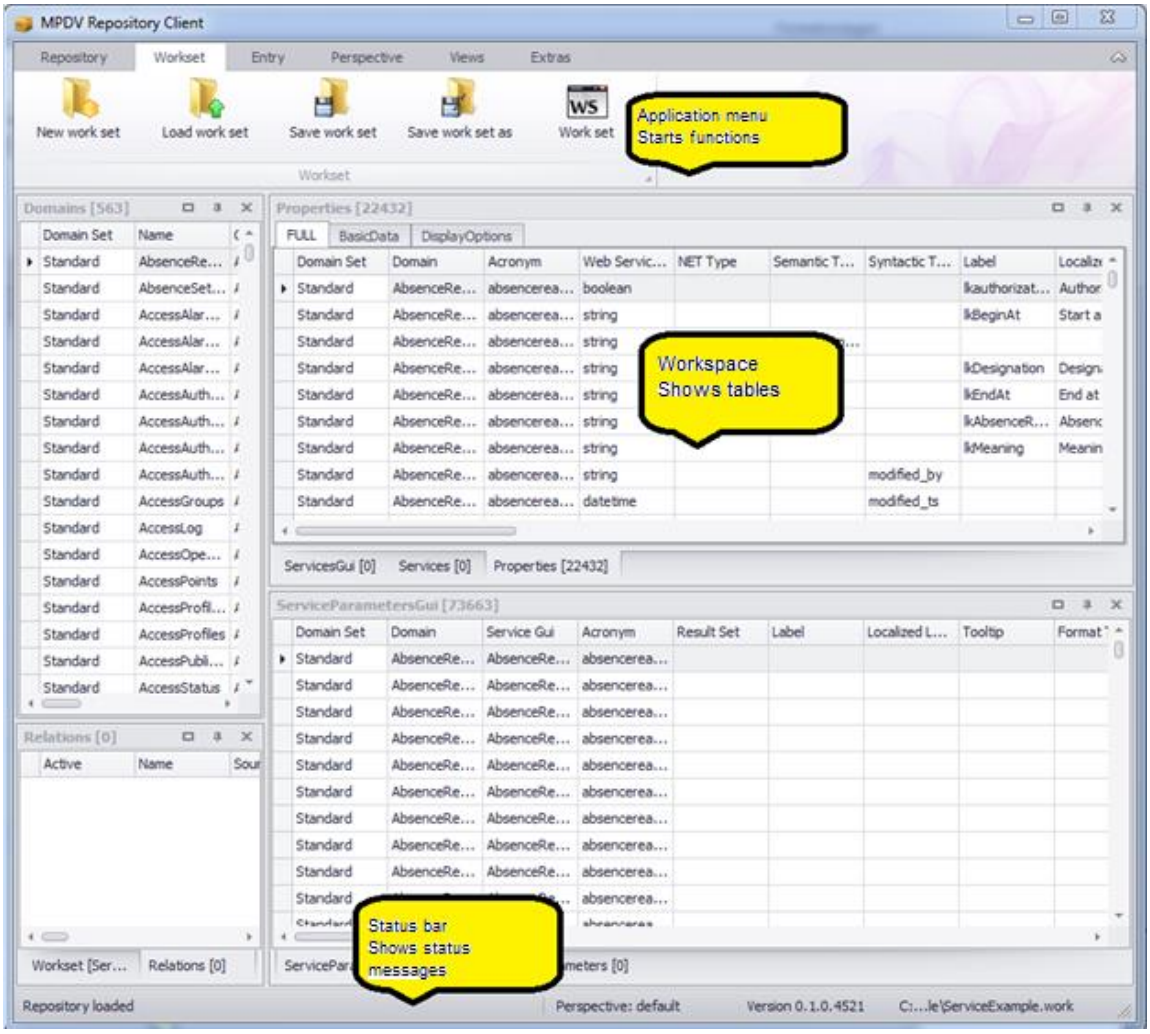
To exit the application, click  in the title bar.

If the loaded repository includes active changes when you exit the application, a respective message is issued asking you to save the changes. If you do not want to save the changes, you can discard the changes or stop exiting the application.

Note: Changes to the workset and perspective are discarded when you exit the application, if you have not saved the changes.

## 3.3 The Application Window

The application window forms a framework for the display of different tables. It includes the application menu with control elements to call and control different functionalities. The menu is on top of the window. A status bar is at the bottom of the window. The status bar shows progress and event messages.




You can individually dock the grids/table views. To do so, click the title bar of a table view/grid and drag it out of the docking position. For orientation purposes, the system shows the docking positions where you can drop the table view. You can also drop a table view without docking it.

## 3.4 Grids/table views

Grids/table views are components to present data records in a table. You can change the tables in the Repository Client according to your requirements. For each grid/table view, the functions described below are available.

The settings, that you make in a table, are saved with the perspective. To undo changes, you can switch to the standard perspective (in the application menu: *Perspective* → *Change perspective*).

**Sort table data**

Click the table header to sort table data in descending order. If you click the table header once more, data is sorted in ascending order. The selected sorting option is shown.

You can sort data by several columns: Press the *Shift* key of your keyboard after sorting the first column. Then click the other column headings by which you want to sort.

You can also use the context menu of the table header to sort data.

**Group data in the table**

You can group table data if the *group by* area is shown. If the *group by* area is not shown, you can show the area via the context menu of the table header (*Show/Hide group by box*). To group by a column, click the column header and drag it to the grouping pane. Multiple grouping is also supported.

**Optimum column width (best fit)**

Select the option *"Best fit"* in the context menu of the table header to adjust the column width of the selected column to the optimum width. In this case, ”optimum” means that the column is as wide as the largest entry in the selected column.

**Optimum column width (all columns) / Best fit (all columns)**

Click this function to adjust all columns to the optimum width.

**Change column width**

You can also change the column width using the mouse, i.e. move the space between two cells to the left or right.

**Show and/or hide columns and entire categories**

Use the context menu function *Select columns* to show and/or hide individual columns and entire categories. For this purpose, select the function in the context menu and then drag the required columns and/or categories from the table to the pool or from the pool to the table.

**Change the sequence of columns and categories**

Also use the mouse to change the display order of columns and categories. To do so, drag the column or category you want to move and drop it at the required location. The system will indicate the location to which the column and/or category will be allocated when you release the left mouse button.

**Freezing columns to prevent horizontal scrolling**

You can freeze columns at the left and right-hand side to keep the columns in view while scrolling. These column settings are included in the perspective and can be saved with the perspective. Right-click the column header and press one of the below-mentioned shortcuts to freeze columns:

- CTRL + right click: freeze at the left-hand side.
- ALT + right click: freeze at the right-hand side.
- SHIFT + right click: Unfreeze.

**Filter table data**

Click the filter icon in the column where you intend to set the filter and select the required filter option from the list; i.e. you select one of the values available in the table or compose a combination of values in the *user-defined* filter.

You can also set several filters in different columns. The table footer indicates that the table has been filtered and also shows the filter criteria. Select the function *Edit filter* on the right of the footer to open the filter editor. Use the filter editor to create complex filter criteria across all columns. You may also open the filter editor via the context menu of the table header.

**Search box**

Use the context menu of the table header to access the option *Show search box.* This option provides a search box within the table. Use this box to quickly search and/or filter the requested data. Simply start typing in this box and the system will only show those rows matching the data you typed. The more characters you enter, the more you narrow down the result.

**Filter row**

Use the context menu of the table header to open the option *Show filter row*. This option provides an additional row shown below the table header. You can enter a search term in any column, and the system will narrow down the displayed rows appropriately. The system supports wildcards. You can also combine search terms in various columns to restrict the search result.

**Edit rows**

Double-click a row or press the "Enter" button to switch to the editing mode and edit the respective row. When you have finished editing and leave the row, the editing mode is terminated. Edited rows are highlighted in color.

## 3.5 The application menu

You can use the application ribbon menu of the Repository Client to control various functions of the tool. It includes several tabs that are described in the following.

### Workset

Includes functions to administer worksets. A workset specifies the sources included in the repository that you want to edit. To display a workset, use the workset panel which consists of a grid/table view.

- **New workset**: This function creates a new workset. If a workset is loaded that has been modified, a dialog pops up asking you to save the changes.
  Click "Yes" to save the changes, "No" will discard them. In both cases, a new workset is created. Click "Cancel" to cancel the process of creating a new workset.
- **Load workset**: This functions loads a workset from an existing file. You can select the workset to be loaded via a file dialog. If the currently loaded workset has been modified, you can save the changes as described above.
- **Save workset**: This function saves the current workset in the file from which it was loaded. If the current workset is new, a file dialog pops up asking you to select the file in which you want to save the workset.
- **Save workset in**: Use this function to save the currently loaded workset in a file. You can select the files using a file dialog.
- **Workset**: Use this button to show and/or hide the grid/table view presenting the currently loaded workset. For details on the workset table, please refer to section Workset.

## Repository

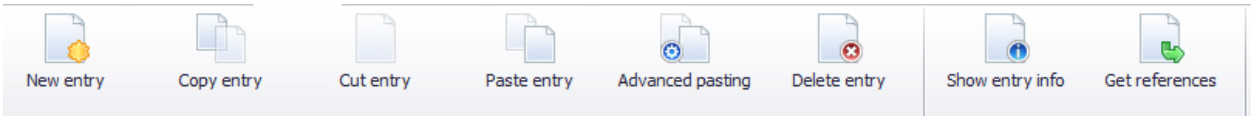
Includes functions to load and save data in the repository. In addition, you may display repository-specific data here.


- **Load repository**: Use this function to load the repository. The currently loaded workset specifies the data sources that are used to load the repository. If a repository is loaded that has been modified, a dialog pops up asking you to save the changes. Click "Yes" to save the changes, "No" will discard them. In both cases, the repository will be reloaded subsequently. Click "Cancel" to cancel the process of loading a repository.
- **Save repository:** ***only available if used in development mode**
  Use this function to save changes in the repository. If no changes have been made, an appropriate note will be displayed.
- **Export repository**: ***only available if used in development mode**
  In contrast to saving the repository, you can use this function to export parts of the repository. For details on this function, please refer to section **Error! Reference source not found.**.
- **Validate**: ***only available if used in development mode**
  You can use this function to validate your data records manually. (See section "Validation").
- **Value list**: Use this menu entry to show and/or hide the value list. The list includes permissible entries for specific fields of the repository.
- **References**: Use this entry to show and/or hide the table with repository references. For details on references, please refer to section References.

- **Changes**: ***only available if used in development mode**
  Use this button to show and/or hide the change view. This view shows the current modifications in the loaded repository.
- **Service documentation**
  Use this button to show the extended documentation of selected standard services. For further information on the service documentation, refer to section "3.9 Service documentation".

## Data collection

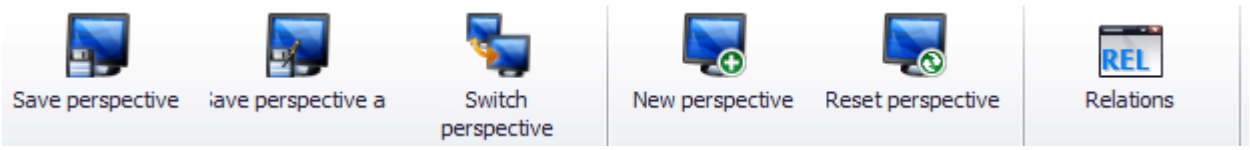
The *Entry* tab summarizes the functions that you can use to edit the loaded repository. The entries refer to the currently focused table view/grid.




- **New entry**: Use this function to create a new entry. For details on this function, please refer to *Context menu* → *New*.
- **Copy entry**: Use this function to copy selected table entries. For details on this function, please refer to *Context menu* → *Copy*.
- **Cut entry**: Use this function to cut selected table entries. For details on this function, please refer to *Context menu* → *Cut*.
- **Paste entry**: Use this function to insert (paste) entries from the cache/clipboard. For details on this function, please refer to *Context menu* → *Insert*.
- **Advanced pasting**: Use this function to edit entries in the clipboard prior to inserting them. For details on this function, please refer to *Context menu* → *Advanced pasting*.
- **Delete entry**: Use this function to delete the selected entries. For details on this function, please refer to *Context menu* → *Delete*.
- **Show entry info**: Use this function to open a dialog showing information on the currently selected entry. For details on this function, please refer to *Context menu* → *Info*.
- **Get references**: Use this function to open a new grid/table view showing the currently selected data record including referenced values. For details on this function, please refer to *Context menu* → *Get references*.

## Perspective

These entries of the menu refer to the administration of perspectives. A perspective is a layout of table views/grids and includes also the associated relations between table views/grids.



- **Save perspective**: Use this function to save the currently shown arrangement of tables.
- **Save perspective as**: Use this function to save the currently shown perspective under a different name. You can enter the new name in the displayed dialog.
- **Switch perspective**: Use this function to change the perspective. A dialog with all available perspectives is shown.
- **New perspective**: Use this function to create a new perspective. Similar to the "Save perspective as" function, you can select the name of the perspective in a dialog. After entering the name, you can immediately switch to the new perspective.
- **Reset perspective**: Use this function to reset the current perspective to the status saved last.
- **Relations**: Use this menu entry to show/hide the grid/table view showing the relations between the grids/table views. For details on relations, please refer to section Relations.

Note: Changes to the perspective are discarded when you exit the application, if you have not saved the changes explicitly.

### Views

Use these entries to open table views/grids. The entries will open a new grid/table view each showing the relevant data records of the repository.

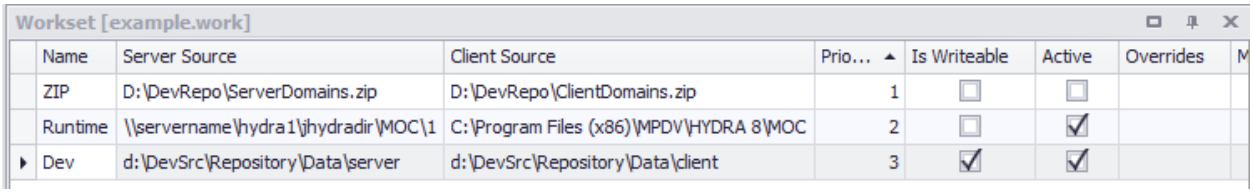

For clear identification of the data records shown in the tables, the *Parent* column is included in each of the tables. The *Parent* column includes the identifier for the father node in the repository tree. The other columns of these tables are defined by the repository documentation.

The *View* area additionally includes a group with entries for the remaining grids/table views of the application.

## 3.6 Workset

Worksets define a set of data sources that make up the repository that you want to edit. Use the workset management function to organize your work on different projects and create an appropriate workset for each of your projects. You can show/hide the workset table via the application menu (*Workset* → *Workset*).

Note: The workset loaded last will be loaded on start of the Repository Client.




The workset table includes the following columns:

**Name**

You can specify the domain set that you want to use to load the data source. The repository data include the information on the domain set that was used to load the data. You can copy data from a domain set into another domain set. Example: Copy existing applications from the domain set "Runtime" (read only) into your development directory, e.g. the domain set "Dev" (writable) where you can make changes.

**Client Source**

You can specify the data source that you want to use to load client data. The following options are available:

- Load data from the runtime structure of the client reference in the server
  In the server, the client configurations are stored as client reference with runtime structure. You can load the repository data from this structure.
  Example:
  HYDRA: x:\jhydradir\MaintenanceManager\rt\client\MOC
  MIP: x:\wsp_config\MaintenanceManager\rt\client\MOC
  The access is read-only.
- Load data from the runtime structure of a local MOC client
  If you enter a path to an MOC installation directory, the respective client data are directly loaded from the MOC runtime installation.
  Example:
  C:\Program Files (x86)\MPDV\HYDRA 8\MOC
  The access is read-only.
- Load data from a local directory with domain structure
  You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.
  Example: d:\DevSrc\Repository\Data\client
- Load data from a ZIP archive
  You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
  The access is read-only.

**Server Source**

You can specify the server source that you want to use to load the server-specific data. The following options are available:

- Load data from the runtime structure in the server
  You can read the configurations for the server in a server installation. To this end, the configuration directory of the web service provider (WSP) up to the instance number is specified.

  Example:
  HYDRA: \\<servername>\<install_dir>\jhydradir\MOC\1
  MIP: \\<servername>\<install_dir>\jdir\MOC\1
  The configuration is loaded from the standard scope by default As an alternative, you can also load the configuration from the *custom* scope, if you enter the value "custom" in the field "name".

  The access is read-only.
- Load data from a local directory with domain structure
  You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.
  Example: d:\DevSrc\Repository\Data\server
- Load data from a ZIP archive
  You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
  The access is read-only.

**Priority**

The priority of a data record specifies the loading sequence of sources that are allocated to the same data record. In the above example, data is first read from the local development directory "d:\DevSrc\Repository" and then from the runtime installations "Server" and "Client". Data records with low priority are overridden and consequently not loaded.

**Is Writeable**

In this column you can specify if a data source grants write access. You only require write access in workset entries where you want to make local developments.

Please note that ZIP archives do not grant write access. For this reasons, you must not enable "is Writeable" in case of a ZIP data source.

Please note that it is not supported to save data in the runtime structure (client directory or server runtime directory). You must therefore not enable "Is Writable" for data sources with

runtime structure.

**Overrides**

You can specify in this column which domain set is overridden by the current one. This affects the resolving of references (for details please refer to section References).

An entry in the "Overrides" column does not have any effect on the loading of the data sources. See column "Priority".

**Active**

Use this option to enable or disable an entry.

## 3.7 Relations

Relations are a property of perspectives and define table filters. Tables that include relations are dynamically adapted to the selected values of another table by setting a filter. For example: Using relations, you can specify that only the service parameters of the service currently selected in another service view are displayed in a service parameter view. The *Relations* table lists the relations of the current perspective. You can call this table via the application menu (*Perspective* →*Relations*).

Relations [13]

| Active | Name | Source | Target | Filter | Var0 | Var1 | Var2 |
|---|---|---|---|---|---|---|---|
| ☑ | Authorizations for Domain | Domains | Authorizations | [Parent] = '$id->Authorizations' | | | |
| ☑ | ControlDataSources for Domain | Domains | ControlDataSources | [Parent] = '$id->ControlDataSources' | | | |
| ☑ | DataObject for Service | Services | Dataobjects | [DomainSet] = '$0' and [Domain] = '$1' AND [Name] = '$2' | DomainSet | Domain | Name |
| ☑ | Properties for Domain | Domains | Properties | [Parent] = '$id->Properties' | | | |
| ☑ | ReferenceData for Domain | Domains | ReferenceData | [Parent] = '$id->ReferenceData' | | | |
| ☑ | SchemaColumns for ServiceParameter | ServiceParameters | SchemaColumns | ([TableName] = '$0' AND [ColumnName] = '$1' AND [TableNam... | DBTabelle | DBField | HydraAcronym |
| ☐ | ServiceParameter for ServiceParameterGui | ServiceParametersGui | ServiceParameters | [Domain] = '$0' AND [Service] = '$1' AND [Acronym] = '$2' | Domain | ServiceGui | Acronym |
| ☑ | ServiceParameters for Service | Services | ServiceParameters | [Parent] = '$id' | | | |
| ☑ | ServiceParametersGui for Service | Services | ServiceParametersGui | [DomainSet] = '$0' and [Domain] = '$1' AND [ServiceGui] = '$2' | DomainSet | Domain | Name |
| ☐ | ServiceParametersGui for ServiceGui | ServicesGui | ServiceParametersGui | [Parent] = '$id' | | | |
| ☑ | Services for Domain | Domains | Services | [Parent] = '$id->Services' | | | |
| ☐ | Service forServiceGui | ServicesGui | Services | [Name] = '$0' | Name | | |
| ☑ | ServicesGui for Domain | Domains | ServicesGui | [Parent] = '$id->ServicesGui' | | | |

**The following columns are displayed:**

**Active**

This checkbox specifies if the relation is used.

**Name**

The name of the relation – free choice.

**Source**

The table and its selection that are used to set the filters. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

**Target**

The table where the filter is applied. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

**Filter**

You can store the filter expression here that you want to apply to the target table. Variables ranging between $0 and $9 are supported. These variables are dynamically filled with the values of the columns Var[0-9].

**Var[0-9]**

In these columns, you can specify the columns of the source table that are used to adapt the filters dynamically.

As soon as a correct (and activated) relation is entered in this table, it is applied. If you close one of the referenced views of a relation, the relevant view is removed from the relation. If this results in a double entry in the *Relations* table, this entry is removed. You can therefore use this view to administer the relations between concrete table instances and to administer unbound relations that can serve as template for relations.

## 3.8 References

References show the inherent connections between data records of the repository. They are defined by the repository structure and cannot be edited in the Repository Client. For example: A value in the column "Syntactic Type" of the *Property* table references another data record in the *Property* table. The *References* table lists the defined references and may be activated via the application menu (*Repository* → *References*).

References [6]

| Name | Source | Source Column | Dependency | Condition | Target | Filter | Priority |
|---|---|---|---|---|---|---|---|
| ControlDataSource1 | Property | ControlDataSource | ControlDataSourceMode | Lookup | ControlDataSource | [Name] = $value and [Domainset] = $domset | -1 |
| ControlDataSource2 | Property | ControlDataSource | ControlDataSourceMode | Reference | ReferenceData | [type] = $value and [Domainset] = $domset | -1 |
| SyntacticType | Property | SyntacticType | | | Property | [Acronym] = $value | 0 |
| SemanticType | Property | SemanticType | | | Property | [Acronym] = $value | 1 |
| Acronym1 | ServiceParameter | Acronym | | | Property | [Acronym] = $value | 3 |
| Acronym2 | GuiServiceParameter | Acronym | | | Property | [Acronym] = $value | 3 |

The following columns are displayed:

- **Name:** Name of the reference
- **Source**: The repository object type that can include this reference.
- **SourceColumn**: The source type property that can include the reference.
- **Dependency**: Source type property specifying the reference target.

- **Condition:** Value that the property specified under *Dependency* must have. Only then, the reference is pursued. For example: The value of ControlDataSourceMode (lookup, reference) specifies the target of the reference (ControlDataSource or ReferenceData) which is specified in the ControlDataSource property.
- **Target**: The repository object type that is referenced.
- **Filter:** Filter that selects the referenced data from the overall quantity of this type of data. Such a filter can include the variable $value (value of column *Value* in the current row) and $parent (value of column *Parent* in the current row).
- **Priority**: Specifies the priority of the reference. You can find further details in section "Get references".

References provide two general functions:

**Show reference**: You can use this function to display the referenced data in a new table. You can call this function via the context menu (*Context menu* → *Show reference*).

Note: This function is only available in the context menu of cells which can include references.

**Get references**: Use this function to complete missing values of a data record with values of referenced data records. For example: You can use this function to show the inherited values of a property of the SemanticType or the SyntacticType.

You can call this function via the context menu (*Context menu* → *Get references*) of the table views/grids. A new data record is generated and shown in a new panel. The generated data record is a copy of the currently selected data record. The values that are not filled are filled by those in the referenced data records. The reference priority specifies the filling sequence.

## 3.9 Service documentation

The Repository Client contains an extended documentation for selected standard services. The documentation mainly includes services released in the system and to be used with the Service Interface (SCS-SIF).

The documentation is available as of MRC version 1.8.STD.65500 (beginning of 2019).

There are two options to access the service documentation:

- In the toolbar "Repository", you can use the button *Service Documentation* to open the table of contents of the services included. Via hyperlinks, you can navigate to the different services.
- In the table views "Services" and "ServicesGui", you can use the context menu to open the documentation of the selected service if it is available.

**Printing a service documentation:**

You can print the service documentation using the shortcut Ctrl-P.